

Parallel Sparse and Data-Sparse Factorization-based Linear Solvers

Xiaoye Sherry Li^[0000-0002-0747-698X] and
Yang Liu^[0000-0003-3750-1178]

arXiv:2602.14289v1 [cs.MS] 15 Feb 2026

Contents

Parallel Sparse and Data-Sparse Factorization-based Linear Solvers	1
Xiaoye Sherry Li ^[0000-0002-0747-698X] and Yang Liu ^[0000-0003-3750-1178]	
1 Introduction	3
1.1 Problem statement	4
1.2 Challenges and motivations	5
1.3 Sources of parallelism	6
1.4 Analysis of parallel algorithms	7
2 Mitigating Communication and Synchronization Costs	8
2.1 Synchronization-avoiding via one-sided communication	8
2.2 Communication-avoiding 3D algorithm framework	10
2.3 Communication lower bounds	13
3 Uncovering Fine-grain Parallelism for GPU Acceleration	15
4 Acceleration via Data-sparse Compression	17
4.1 Overview of rank-structured matrix algorithms	18
4.2 Existence of hierarchical matrix structure in practical applications	19
4.3 Algorithm description with a $\mathcal{H}^2$ matrix example	21
4.4 Parallelization of rank-structured matrix algorithm	25
5 Combining Structure-sparse and Data-sparse Formats	27
5.1 Overview of data-sparse multifrontal sparse solver and preconditioner	28
6 Available Software for Distributed Memory Parallel Computers	31
7 Open Problems	33
References	33
References	33

Abstract Efficient solutions of large-scale, ill-conditioned and indefinite algebraic equations are ubiquitously needed in numerous computational fields, including multiphysics simulations, machine learning, and data science. Because of their robustness and accuracy, direct solvers are crucial components in building a scalable solver toolchain. In this article, we will review recent advances of sparse direct solvers along two axes: 1) **reducing communication and latency costs in both task- and data-parallel settings**, and 2) reducing computational complexity via low-rank and other compression techniques such as hierarchical matrix algebra. In addition to algorithmic principles, we also illustrate the key parallelization challenges and best practices to deliver high speed and reliability on modern heterogeneous parallel machines.

1 Introduction

Efficient solutions of large-scale, ill-conditioned and indefinite algebraic equations are ubiquitously needed in numerous computational fields, including multiphysics simulations, machine learning, and data science. Because of their robustness and accuracy, direct solvers are crucial components in building a scalable solver toolchain. In this article, we will review recent advances of sparse direct solvers along two axes: 1) reducing communication and latency costs

in both task- and data-parallel settings, and 2) reducing computational complexity via low-rank and other compression techniques such as hierarchical matrix algebra. In addition to algorithmic principles, we also illustrate the key parallelization challenges and best practices to deliver high speed and reliability on modern heterogeneous parallel machines.

1.1 Problem statement

A linear system is defined as $Ax = B$, where matrix A is of size $n \times n$ and the right-hand side B is of size $n \times nrhs$. This chapter focuses on sparse linear systems in two scenarios. **In the first case A is structure-sparse with many explicit zeros. This type of sparse systems typically arises from discretization of partial differential equations. In the second case A is dense but admits data-sparse representations**, for example, it is numerically low-rank or has numerically low-rank blocks. This type of sparse systems often arise from integral equations, boundary value problems and many kernel matrices used in machine learning. Even though the solution techniques for the two types of sparse systems are distinctively different, we will illustrate that a hybrid method combining both types of sparsity-exploiting techniques is even more efficient than a traditional sparse direct solver. **For the right-hand side, we focus on the most common case with dense B and $nrhs = 1$.**

In this chapter, by “direct solver” we mean more broadly the solvers or preconditioners that are based on certain matrix factorizations. That is, the core operations are factorizations and solutions using the factored matrices. **In data-sparse cases, the factorizations are usually approximate, therefore they may not be used simply in direct solvers, instead, they can be used as effective preconditioners to accelerate convergence of a Krylov iterative solver.** We note that the low-rank approximate factorization is very different from the conventional ILU-type approximate factorization, which will be clear in section 4.

We will focus on two main algorithmic advances in recent years: 1) asymptotically reduce data movement on parallel machines, and 2) asymptotically reduce arithmetic operations and memory footprint. Here, by asymptotic reduction we mean the lower exponent in “big O” notation in analytical formulae. For example, in the first case, we will show orders of magnitude lower in communication bandwidth and latency cost for some algorithms. In the second case, we will show almost linear complexity for arithmetic and memory for some algorithms. The complexity formulae will be summarized in tables 1 to 4. Guided by these algorithmic principles, a large number of implementation techniques were developed. We will focus on the distributed-memory algorithms for both CPUs and GPUs, targeting exascale.

For most practical problems, we first *transform* the original linear system into a more favorable form that can be solved more efficiently. A general description of the algorithm for direct solution of linear equations $AX = B$ is as follows:

1. Compute a *factorization*, for example, LU decomposition:

$$P_c P_r D_r A D_c P_c^T = LU \quad (1)$$

Here D_r and D_c are diagonal matrices to equilibrate the system, which can often reduce the condition number. P_r and P_c are *permutation matrices* that reorder the rows and the columns of A . They serve the purpose of maintaining numerical stability and preserving sparsity in the sparse factored matrices.

2. Compute the *solution* of the transformed system: $(P_c P_r D_r A D_c P_c^T) P_c D_c^{-1} X = P_c P_r D_r B$. Thus, X can be evaluated as:

$$X = A^{-1} B = D_c P_c^T U^{-1} L^{-1} (P_c P_r D_r B) \quad (2)$$

Multiplying L^{-1} (or U^{-1}) means triangular solution in the case of LU decomposition. Or, it could mean some other solution methods for other types of factorizations, e.g., see section 4.

Remark. Computing D_r , D_c , P_r and P_c is considered a preprocessing step, which by themselves have large bodies of literature. Although this is not the focus of this chapter, they are critical components for a robust and efficient solver and we will give appropriate references throughout the chapter, see, for example, [128].

Most of the algorithms discussed in this chapter are based on Gaussian elimination (GE) and its sparse variants. In the GE algorithm the Schur complement updates can be performed in different orders, leading to different variants such as left-looking (fan-in) or right-looking (fan-out). These variants are mathematically equivalent under the assumption that the operations are associative, although this is not entirely true in floating point arithmetic. However, different variants

Listing (1) right-looking LU	Listing (2) symmetric-pattern multifrontal LU
For each node K in DAG from sources to sink	for each node K in etree from leaves to root
1. Factorize diagonal $A(K, K) = L(K, K)U(K, K)$ (may involve pivoting) 2. Compute off-diag panels $L(R_1, K)$, $U(K, R_1)$ (via triangular solves TRSM) 3. In-place Schur complement update: $A(R_1, R_1) \leftarrow A(R_1, R_1) \leftarrow (-L(R_1, K) \cdot U(K, R_1))$ (via matrix multiplication GEMM)	0. Assemble children's Ts to frontal matrix: $F \leftarrow \begin{bmatrix} A(K, K) & A(K, R_1) \\ A(R_1, K) & 0 \end{bmatrix} \leftarrow T_1 \leftarrow T_2 \leftarrow \dots$ 1. Factorize diagonal $F(K, K) = L(K, K)U(K, K)$ (may involve pivoting) 2. Compute off-diag panels $L(R_1, K)$, $U(K, R_1)$ (via triangular solves TRSM) 3. Compute and pass update T_K to parent: $T_K = F(R_1, R_1) - L(R_1, K) \cdot U(K, R_1)$ (via matrix multiplication GEMM)
Endfor	Endfor

Fig. 1: Sketch of the right-looking and multifrontal algorithms. We define $R \subseteq \{K : N\}$ consisting of indices corresponding to nonzeros of LU, and $R_1 = R \setminus \{K\}$. In Listing (2), T_i represents node i 's update matrix corresponding to the partial Schur complement update. The two index sets R and R_1 change after each step of GE, reflecting the Schur complement sparsity after each elimination step.

have very different memory access and communication patterns. We mainly use the right-looking algorithm and its multifrontal variant to illustrate parallelism. Figure 1 gives a high level sketch of the two algorithms and compare them side-by-side. In the algorithms a node usually refers to a *supernode*. Loosely speaking, a supernode consists of several consecutive rows/columns having the same nonzero pattern. Supernodes help lift most of the submatrix operations to use Level-3 BLAS, leading to higher arithmetic intensity.

Remark. The multifrontal algorithm presented here works only for sparse matrices with *symmetric patterns*, hence, the computation can be organized in a tree structure, called assembly tree or elimination tree (etree) [144]. For nonsymmetric-pattern sparse matrices, the computation graph is directed acyclic graph (DAG), the same as the right-looking or left-looking algorithm. See more explanation of a DAG in section 2.1. A good example of nonsymmetric-pattern multifrontal algorithm and code is UMFPACK [64].

In the algorithm description, we use the multifrontal term *extend-add*, with symbol $\leftarrow \oplus$, to denote the sparse operation of *scattering-and-adding* an element from the source array into an element of the destination array. The nonzero structure of the destination is a superset of that of the source, hence the need for “scatter”. Note that in the context of Schur complement update, the “add” operation should really be interpreted as “subtract”. For notational convenience, we define two index sets $R \subseteq \{K : N\}$ referring to the remaining part of the matrix at the K th elimination step, and $R_1 = R \setminus \{K\}$. These two index sets change after each step of GE, depending on sparsity. In the supernodal right-looking algorithm (Listing (1) in fig. 1), steps 1 and 2 involve dense matrix operations. Step 3 involves sparse scattering. In the multifrontal algorithm (Listing (2) in fig. 1), steps 1 and 2 are the same as the right-looking algorithm. The difference is in the treatment of Schur complement update. In right-looking, the Schur complement update is performed right away *in-place*, whereas in multifrontal, multiple partial Schur complement update matrices (denoted as T) are passed along the etree. The scatter operation occurs at step 0.

In [17], Amestoy et al. provided a comprehensive study and comparison of these two algorithms that were implemented in the distributed memory SuperLU-DIST [199] and MUMPS [18], respectively. Empirically, using a 512-processor Cray T3E and a set of matrices from real applications, the paper compared the performance characteristics of the two solvers with respect to uniprocessor speed, interprocessor communication, the memory requirement, and scalability.

The GE algorithms involving hierarchically low-rank matrices will be treated in section 4.

1.2 Challenges and motivations

Parallel computing has become an increasingly indispensable tool in various computing disciplines, particularly in modeling physical phenomena in science and engineering simulations. Developing parallel algorithms and software has

become a central task in scientific community. Despite a long history of advances in parallelizing sparse linear algebra algorithms, there still exist many challenges, either architecture-related or application-related, which are summarized below.

1. Inter-process communication cost and load imbalance become an increasing scalability hindrance. This is due to the growing gap in hardware speed of data transfer and the speed of floating point operation. Therefore, it is essential to redesign algorithms to reduce communication and synchronization. We address this challenge in section 2.
2. Modern HPC architectures become increasingly heterogeneous. For the three exascale machines in the United States, more than 90% of the computing power comes from GPUs. Existing sparse direct solvers usually exploit only task parallelism, e.g., through OpenMP¹ to use on-node CPUs and MPI² to communicate across nodes. Novel methods are needed to make best use of massive data parallelism provided by a GPU. Worse yet, no common programming standard is developed for GPUs from different computer vendors, e.g., AMD, NVIDIA, Intel. We address this challenge in section 3.
3. Direct solvers based on classical LU decomposition and triangular solution are inherently not scalable from the algorithmic complexity viewpoint, because the amounts of floating point operation and memory footprint grow super-linearly with respect to the problem size. The growth rate is even higher for the discretized PDEs in 3D geometry than the 2D ones. Put it another way, the number of computing nodes must increase more than the increase of problem size in order to be able to store the factored matrices in the aggregate memory of all the nodes. This limits the problem size that are solvable given the fixed hardware resources and time budget. A truly scalable algorithm should have a nearly linear time and memory complexity with respect to the problem size. We address this algorithmic challenge in section 4.

1.3 Sources of parallelism

Parallel architectures

Parallelism occurs at many levels of hierarchy of modern processor architectures with varying granularity: bit level, instruction level, data, and task parallelism [103].

The vector machines with SIMD (Single Instruction, Multiple Data) instructions were the earliest parallel machines which are based on the pipeline principle. They provide high-level operations on arrays of elements. SIMD instructions perform exactly the same operations on multiple data objects, thus produce multiple results at the same time. For example, in late 1990s, Intel introduced the SSE (Streaming SIMD Extensions) (and SSE2 extension for double precision) in the x86 instruction sets, which were later expanded to MMX, AVX, and AVX512. For the SIMD architectures, parallelization is usually achieved by a compiler which analyzes the program and generates the vector instructions. However, the SIMD instructions are not friendly with operations with irregular control structures.

The growth of multi-core CPUs with independent cores that can handle distinct threads of execution serves a much broader spectrum of applications that can take advantage of parallel computing. These so-called Multiple Instruction Multiple Data (MIMD) machines are suitable for general-purpose computing tasks, including servers, and high-performance computing (HPC). In the MIMD architectures, shared memory machines are widely used now on desktops and laptops. Multiple processors access a common memory space and communicate through this shared memory. Programming such systems are made easier by using the standard OpenMP API that is portable to many platforms and programming languages. Distributed memory machines are needed to provide true scalability, where each processor has its own local memory. Processors communicate via a interconnect network. Programming such systems is significantly harder than for a shared memory machine. The MPI standard brought a portable solution for message passing which quickly became the de facto standard. Nowadays, most HPC systems are of hybrid architectures which combine elements of both shared and distributed memory systems. Additionally, each node often has heterogeneous organization with both multicore CPUs and GPU accelerators. It is insufficient to use MPI alone to program such systems. We often use a hybrid programming model such as MPI+X, where MPI is used to generate multiple processes across multiple nodes, and X is used within the address space of each MPI process. Here, X can be OpenMP and/or GPU-specific languages.

¹ <https://www.openmp.org/>

² <https://www.mpi-forum.org/>

Parallel algorithms in matrix computation

Parallelization is to divide the whole computational task into multiple subtasks and map them to the underlying architecture. The art of designing an efficient parallel algorithm follows several key guiding principles: 1) dividing the subtasks evenly to maintain balanced workload across all the processes; 2) making subtasks as independent as possible to avoid information sharing and hence the waiting time; 3) increasing arithmetic intensity relative to memory access and interprocess communication; and 4) using high-level abstraction.

Matrix computation is one of the prime fields that could effectively utilize parallel computers at the dawn of parallel computing. Even though it is a mature field in parallel algorithms design, there are still ample opportunities for algorithm advances, this is mainly due to the frequent innovations of HPC architectures and novel applications that rely on numerical linear algebra. Over years, the single most important abstraction is the Basic Linear Algebra Subprograms (BLAS) library³ which contains low-level routines that perform basic vector and matrix operations, providing essential building blocks for linear algebra applications. Various optimized BLAS libraries are widely accessible, such as OpenBLAS, ATLAS and Intel MKL for CPUs, and NVIDIA cuBLAS and AMD rocBLAS for GPUs. The BLAS abstraction hides a lot of low level architectural optimization from the algorithm developers, such as vectorization or threads/warps on GPUs.

In sparse matrix landscape, many algorithms are also restructured to express the operations in terms of BLAS. However, the dense blocks in sparse matrices are usually small, making an algorithm far from performing at the peak speed if it depends only on parallelizing BLAS. Therefore, in addition to data parallelism encompassed in BLAS, it is essential to exploit task parallelism as well. In sparse factorization-based solvers, two important data structures can greatly help design a good matrix-to-process mapping scheme and a parallel execution strategy. One data structure is DAG as is used in the right-looking sparse LU. Another data structure is tree as is used in the symmetric-pattern multifrontal LU and the hierarchically rank-structured algorithms in section 4. Both DAGs and trees encode the data and task dependencies among submatrix operations. There is a rich body of literature about both static and dynamic scheduling of a task graph described by a DAG or tree on a distributed memory machine to maintain load balance and to minimize communication. The parallel algorithms presented in this chapter are mostly based on static scheduling strategies. The challenge with dynamic scheduling is the overhead associated with the scheduler itself. It is cost effective only when the computational task has large granularity.

1.4 Analysis of parallel algorithms

We emphasize established performance bounding models.

- *Amdahl's law* provides an upper bound on achievable speedup based on the fraction of sequential work, highlighting the diminishing returns of parallelization when sequential portions remain.
- The *roofline model* helps relate algorithmic arithmetic intensity to architectural bandwidth and peak performance, serving as a visual performance diagnostic.
- The *latency-bandwidth model* is discussed to quantify the per-message startup overheads and sustained transfer rates, guiding optimizations in communication patterns.

The rest of the chapter is organized as follows. Section 2 describes the recent algorithms for mitigating communication cost for sparse LU factorization (SpLU) and sparse triangular solution (SpTRSV). This section also surveys analyses of the communication lower bounds for these operations. Section 3 describes various novel algorithms to exploit the massive fine-grain parallelism provided by the GPUs. Section 4 describes explores various data-sparse compression techniques to reduce both floating point and memory complexity. In addition to the parallel construction algorithms for compression, the section also illustrates how the GE algorithm works with the compressed formats and how they can be efficiently parallelized. Section 5 describes how the data-sparse formats can be embedded in the structure-sparse algorithm, leading to a nearly linear complexity fast solver. Section 6 gives a brief survey of the existing parallel sparse solvers, using one or multiple algorithms discussed in this chapter. Finally, section 7 lists a number of open problems.

³ <https://netlib.org/blas/>

2 Mitigating Communication and Synchronization Costs

Communication cost can occur when transferring data between fast and slow memory or between two nodes across the network. This chapter focuses more on the latter. A commonly used cost model for communication performance is α - β *model*, where α refers to the latency and β is the inverse of the bandwidth between two processes. The time to send a message of length n is roughly:

$$\text{Time} = \text{latency} + n/\text{bandwidth} = \alpha + n \times \beta .$$

This is a simplified and ideal model, without taking into account such practicalities as network congestion etc. For most parallel machines $\alpha \gg \beta \gg \text{time_per_flop}$. For example, on the current Permuter node at NERSC ⁴, typical sparse matrix algorithms run at about 4% of the peak, i.e., realistic time per flop is $10^{-5} \mu s$. According to the measurement by Lin [171] of typical message sizes, $\alpha = 1.7 \mu s \approx 170,000$ flops, and $\beta = 4 \cdot 10^{-5} \mu s/\text{byte} \approx 30$ flops_per_double_word. Therefore, the fundamental principle is to organize the parallel algorithm so that it maintains high data locality and sends fewer long messages rather than sending many short messages. In the following subsections we will describe various algorithmic and implementation techniques to mitigate communication cost for SpLU and SpTRSV.

2.1 Synchronization-avoiding via one-sided communication

Sparse computations in general have lower arithmetic intensity than the dense counterparts. The problem is exacerbated by a high degree of data and task dependency. For example, sparse triangular solution (SpTRSV) is notoriously difficult to parallelize – its arithmetic intensity is of $O(1)$, similar to sparse matrix-vector multiplication (SpMV). Yet, it has higher degree of dependency than SpMV, which imposes more sequentiality than SpMV.

The execution of factorization and triangular solution can be concisely prescribed by a data structure called Directed Acyclic Graph, or DAG, in which a vertex i represents certain computation task T_i and a directed edge from vertex T_i to vertex T_j encodes the dependency of T_j on T_i , e.g., T_j needs some data produced by T_i . The vertex weight represents the amount of computation and the edge weight represents the amount of data to be transferred between two tasks. If an edge crosses the boundary between two MPI processes, inter-process communication needs to happen for data transfer.

The DAGs for sparse computations can be highly irregular in that both the vertex weights and the edge weights vary a lot. The communication links between pairs of processes also vary. It is particularly challenging to achieve good parallel efficiency for SpTRSV, which typically has limited task-level parallelism and relies on fine-grain synchronization to exploit this parallelism.

For SpTRSV on-node parallelism, extensive studies have led to efficient algorithms with satisfactory parallel efficiency. All these algorithms aim to schedule ready tasks as fast as possible, including level-set based methods [21, 145], level tile [207] and dynamic scheduling [46]. Figure 2 shows an example sparse lower triangular matrix L and its associated task graph. Assuming a supernodal blocked algorithm is used, then each diagonal block L_{ii} defines a dense TRSV operation $x_i = L_{ii}^{-1} \bar{b}_i$ and each off-diagonal nonzero block L_{ij} defines a dense GEMV operation $L_{ij} x_j$. In a level-set algorithm, multiple processes perform parallel operations within a level, then synchronize globally before moving to the next level. Park et al. [180] developed a scheme to reduce the amount of synchronization. The algorithm uses local point-to-point synchronization for the two dependent tasks across the level sets if they are assigned to different processes. Dependent tasks assigned to the same process do not incur synchronization. They showed that this reduced synchronization led to 1.6 $\times$ speed up over the strictly level-based algorithm on a 12-core Intel Xeon processor.

Implementing these techniques in a distributed manner is highly nontrivial, since the algorithms do not map well to MPI's two-sided handshaking paradigm, where both sender and receiver actively engage in the data exchange. In contrast, one-sided communication is an appealing alternative communication model, which was already established in the MPI standard since MPI-2. In one-sided communication paradigm communication between processes does not require explicit synchronization between the sender and receiver. Instead, one process can perform operations on the memory of another process without the receiving process's explicit participation. MPI-2 introduced the MPI_WIN and Remote Memory Access (RMA) operations to take advantage of the remote direct memory access (RDMA) features offered by modern interconnects RDMA enables a process to directly access memory on remote processes without

⁴ <https://www.nersc.gov/what-we-do/computing-for-science/perlmutter>

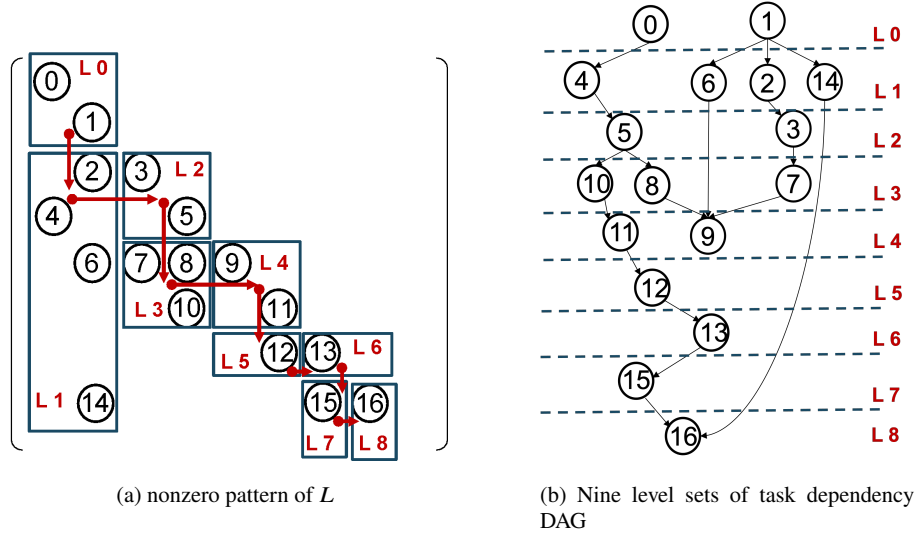


Fig. 2: Illustration of level set in a lower triangular SpTRSV, $Lx = b$.

involvement of the activities at the remote side. The light-weight asynchronous one-sided communication primitives provide a new mechanism for efficient DAG execution at scale.

The first distributed-memory SpTRSV algorithm in SuperLU_DIST uses a *message-driven* approach [129]. Every process runs an asynchronous self-scheduling loop and triggers appropriate computations depending on one of the two types of messages received. This algorithm overcame some synchronization overhead, but still uses MPI two-sided Isend/Recv communication protocol. It can hardly strong scale to 100 processes. The algorithm was improved by Liu et al. [150] by using asynchronous communication trees for collective communication among subsets of processes.

In [68], Ding et al. developed the first SpTRSV algorithm exploiting the one-sided communication paradigm. Firstly, they benchmarked a couple of one-sided MPI implementations on a Cray XC40 parallel machine with Intel Xeon E5-2698v3 processors, one is Cray MPI [112] and the other is foMPI [34]. The latter is a fast MPI-3.0 RDMA library on the Cray system. They found that on 16 nodes, the foMPI library achieved up to $3\times$ more bandwidth than the Cray MPI two-side primitives for messages of size 64 to 256 bytes, which is the typical size in SpTRSV. Secondly, they profiled the earlier distributed SpTRSV code in SuperLU_DIST with communication tree optimization [150] and found that with 1000 MPI processes, as much as 90% of the parallel runtime is spent in communication.

Ding's algorithm uses a *passive target mode* in one-sided communication to implement two synchronization-free task queues to manage the messaging between producer-consumer pairs. One such task queue is dedicated to the solution components x_i 's after they are computed at the diagonal blocks and are remotely enqueued, via MPI.Put, to the column processes that need them for subsequent matvecs. Another task queue is dedicated to the partial sums of the matvecs at the diagonal blocks, via MPI.Accumulate. They empirically showed that, using up to 4096 processes on the Cray XC40 system, their one-sided SpTRSV solver reduces communication time by up to $2.5\times$ and improves runtime up to $2.4\times$ compared to the SuperLU_DIST's two-sided MPI implementation.

Another contribution of Ding's work is the development of a critical path performance model for the DAG-based computational graph of SpTRSV. Ding showed that this performance model can accurately assess the observed performance relative to computation and communication speeds and provide advice on what shape of the process grid is optimal [68]. This model is generalizable to any parallel applications with DAG as an underlying computation/communication graph.

Given the delicate task queue algorithm design and careful implementation, a natural question to ask is whether a genetic DAG-based runtime system can achieve the same effect. In the last two decades, there has been tremendous progress in development and deployment of runtime frameworks for parallel algorithms portable to heterogeneous architectures. The two widely used systems in linear algebra community are PaRSEC [41] and StarPU [26]. These two runtime systems were successfully used in parallel implementations of SpLU [125, 186]. However, it remains an open problem whether such generic runtime scheduler is cost-effective for SpTRSV, because the task sizes in SpTRSV are much smaller than those in SpLU. The only work we are aware of in this vein is by Totoni et al. [202], where

the message-passing parallel language and runtime system Charm++ [111] was used for load balancing and dynamic scheduling. They showed that the Charm++ assisted implementation can scale up to 512 core of an IBM BlueGene/P machine.

To extend the one-sided communication paradigm to the GPU realm, Ding et al. leveraged the advantage of GPU-initiated data transfers of NVSHMEM to implement and evaluate a multi-GPU SpTRSV [67]. Ding’s SpTRSV algorithm is based on a novel producer-consumer paradigm to manage the computation and communication using two separate CUDA streams. On two nodes of NVIDIA Tesla V100 GPUs, the algorithm achieved a $3.7\times$ speedup when using two nodes of twelve GPUs relative to the baseline on a single GPU, and up to $6.1\times$ compared to cuSPARSE csrsv2() over the range of one to eighteen GPUs.

2.2 Communication-avoiding 3D algorithm framework

As the speed discrepancy between communication and floating point operation becomes increasingly large, communication time often dominates the parallel runtime of a parallel algorithm, especially for a sparse matrix algorithm. One primary innovation of the parallel sparse direct solver in the 1990s and 2000s was the 2D blocking algorithms for sparse LU decomposition and sparse triangular solution, such as SuperLU_DIST [129, 130] and WSMP [97]. This was motivated from the dense algorithms design in the ScaLAPACK library [38]. In these 2D algorithms, the processes are organized in a two-dimensional grid, with communication restricted within process rows or process columns. The sparse L and U factors are partitioned into 2D blocks, which generally are of non-uniform size and are sparsity-induced. These 2D blocks are cyclically mapped to the 2D process grid. However, when using thousands processes, more than 90% time was still spent in communication [192].

In recent two decades, there is a growing body of research on novel algorithms that require asymptotically lower amount of communication than the existing parallel algorithms (e.g., [27]). These algorithms are termed communication-avoiding algorithms, or CA algorithms for short. One notable class of CA algorithms is to trade off extra memory for reduced communication. The rationale behind this is based on the analysis that the communication lower bounds decrease as the size of local memory increases [27, Section 2]. Based on this principle, the 3D and 2.5D CA algorithms were developed for dense matrix-multiplication and LU decomposition [110, 196].⁵

Inspired by the success of 2.5D dense CA algorithms, Sao et al. developed the first 3D CA algorithm for SpLU [192] and SpTRSV [188]. The 3D CA algorithm framework uses a 3D logical process grid, instead of the 2D process grid that was the state-of-the-art as of 2018. The algorithm replicates some data to reduce both the number of messages and the communication volume. In addition, the 3D sparse algorithms can use the elimination tree to efficiently map the sparse matrix to the 3D process grid to further localize the communication among subsets of processes. On the other hand, the 2.5D dense LU algorithm cannot exploit this feature. As a result, the 3D sparse algorithm not only reduces communication but also shortens the critical path of SpLU. For matrices with planar graph structure (e.g., planar grids and meshes), the critical path length of Sao’s 3D SpLU algorithm is $\mathcal{O}(n / \log n)$ whereas the critical path of a typical 2D algorithm is $\mathcal{O}(n)$.

Figure 3 illustrates the 3D processes grid, configured as $P = P_x \times P_y \times P_z$. It can be considered as P_z layers of 2D processes. The distribution of the sparse matrices is governed by the supernodal elimination tree: the standard etree is transformed into an etree which is binary at the top $\log_2(P_z)$ levels and has P_z subtrees at the bottom level. The transformation may make some subtrees disconnected, hence becoming forests. But for notational simplicity, we still use the term etree herein. The matrices A , L , and U corresponding to each subtree are assigned to one 2D process layer. The 2D layers are referred to as Grid-0, Grid-1, ..., up to $(P_z - 1)$ grids. Figure 4 shows the submatrix mapping to the four 2D process grids. The sparse LU on each 2D grid is performed using the same 2D algorithm locally within the 2D processes. Each of the 2D LU generates a Schur complement update to the common ancestor of the etree. The key feature of the 3D LU is to replicate the Schur complement memory corresponding to the common ancestor tree among the P_z 2D process layers, so that each 2D grid can accumulate the Schur complement (partial sum) in their local memory, in parallel with the other 2D grids. Finally, the individual copies of partial sums are reduced (added) onto a single grid, where it is factorized in a 2D fashion. The communication reduction is mainly due to the independent 2D subgrids working on smaller subproblems. For example, in fig. 4a, let S_i^j denote the partial update of the Schur complement that is computed and residing in the process Grid-j but is destined to the etree node i . At the leaf level,

⁵ The 3D process configuration is a special case of the 2.5D configuration.

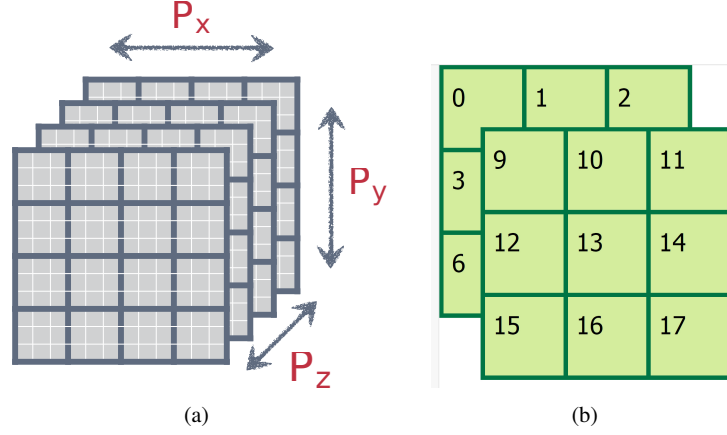


Fig. 3: The view of the logical 3D process grid and an example of 18 processes arranged as a 3x3x2 process grid.

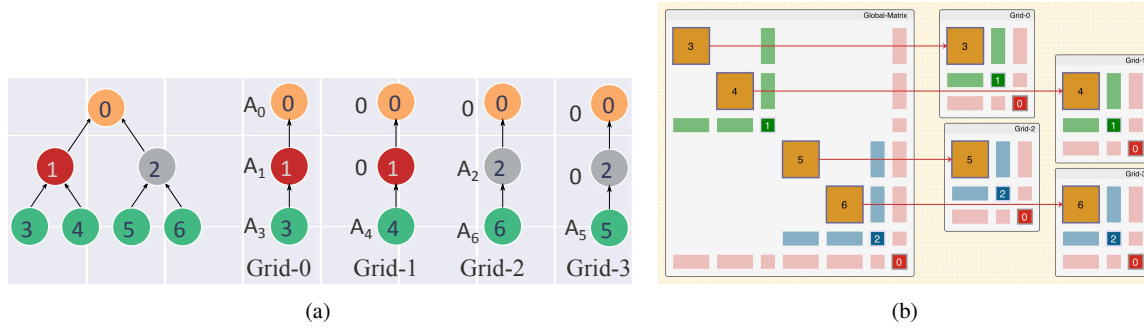


Fig. 4: Two-level etree partition and the matrix view of the submatrix mapping to four 2D process grids.

Grid-0 and Grid-1 first factorize nodes 3 and 4 in parallel. Grid-0 produces the partial updates S_1^0 and S_0^0 . Grid-1 produces the partial updates S_1^1 and S_0^1 . Then the two grids reduce all the common ancestor nodes, namely nodes 1 and 0, to grid-0. This amounts to subtracting $S_1^0 + S_1^1$ from A_1 and subtracting $S_0^0 + S_0^1$ from A_0 . In parallel, similar operations occur for Grid-2 and Grid-3 along the etree nodes 5, 6, 2 and 0. At one level up, only Grid-0 and Grid-2 are active. They factorize nodes 1 and 2, producing S_0^0 and S_0^2 , and reduce the updates on node 0 to Grid-0. At the root level, only Grid-0 factors node 0. The generalization of this process for any $P_z = 2^l$ is given as Algorithm 1 in [192]. In practice, the 3D SpLU algorithm improves the strong scaling from 1000 cores to 32,000 cores, leading to speedups of up to 27 $\times$ on planar and up to 3.3 $\times$ on non-planar sparse matrices.

For a sparse matrix with arbitrary sparsity pattern, it is infeasible to quantify the communication reduction in "big O" sense. Fortunately, for a large class of discretized prototypical PDEs, there exist analytical closed forms of nonzero patterns in the LU factors. These include the finite difference discretization of the PDEs defined on the 2D geometry giving rise to planar graphs, and the PDEs defined on the 3D geometry giving rise to non-planar graphs. Sao et al. analyzed the memory-communication trade-off for these classes of problems. They assume that the matrices are first reordered by the nested dissection ordering [83] or recursive graph partitioning [142, 88], which is optimal for this class of problems [83, 142, 88]. With this ordering, the number of nonzeros (memory) in the LU factors are $O(n \log n)$ for planar graphs and $n^{4/3}$ for non-planar graphs. The flop counts are $n^{3/2}$ for planar graphs and n^2 for non-planar graphs. The etree corresponds to the *separator tree* where each internal node is a separator at certain middle level of the dissection. The memory and communication analyses use the following two key facts:

- Each separator node in the etree corresponds to a dense submatrix;
- Communication time is determined by the amount of communication occurring on the *critical path* of the computational graph. Here, the critical path is a leaf-to-root path in the separator tree.

Table 1 summarizes the analysis results regarding the memory, bandwidth and latency costs, comparing the 2D and 3D sparse LU algorithms. The analysis calculates the memory and communication in two separate phases — first in

the 2D algorithm for each subtree and second in the ancestor-reduction step. Thus, in the column “3D-SparseLU”, two separate terms contribute to memory and communication. The calculation is only needed for Grid-0 as it is the only grid that participates in all the levels and is on the critical path. The last column suggests that setting $P_z = O(\log n)$ would lead to the minimal memory and communication costs for the 3D algorithm, assuming $P \gg \log n$. Similarly, table 2 summarizes the memory, bandwidth and latency costs for the PDEs associated with the non-planar graph.

Table 1: [192, Table 2] Asymptotic memory, communication, and latency costs for 2D and 3D Sparse LU algorithm for 2D model PDEs.

Parameter	2D PDE		
	2D-SparseLU	3D-SparseLU	3D-SparseLU $P_z = O(\log n)$, minimum
FLOP count			
per process (F)	$O\left(\frac{n^{3/2}}{P}\right)$	$O\left(\frac{n^{3/2}}{P}\right)$	$O\left(\frac{n^{3/2}}{P}\right)$
Memory			
per process (M)	$O\left(\frac{n}{P} \log n\right)$	$O\left(\frac{n}{P} \left(\log\left(\frac{n}{P_z}\right) + P_z\right)\right)$	$O\left(\frac{n}{P} \log n\right)$
Communication volume			
per process (W)	$O\left(\frac{n}{\sqrt{P}} \log n\right)$	$O\left(\frac{n}{\sqrt{P}} \left(2\sqrt{P_z} + \frac{\log n}{\sqrt{P_z}}\right) + \frac{2nP_z \log P_z}{P}\right)$	$O\left(\frac{n}{\sqrt{P}} \sqrt{\log n}\right)$
Latency	$O(n)$	$O\left(\frac{n}{P_z} + \sqrt{n}\right)$	$O\left(\frac{n}{\log n}\right)$

Table 2: [192, Table 3] Asymptotic memory, communication, and latency costs for 2D and 3D Sparse LU algorithm for 3D model PDEs.

Parameter	3D PDE	
	2D-SparseLU	3D-SparseLU
FLOP count		
per process (F)	$O\left(\frac{n^2}{P}\right)$	$O\left(\frac{n^2}{P}\right)$
Memory		
per process (M)	$O\left(\frac{n^4}{P}\right)$	$O\left(M_{2D} \left(P_z + \frac{1}{P_z^{4/3}}\right)\right)$
Communication volume		
per process (W)	$O\left(\frac{n^{4/3}}{\sqrt{P}}\right)$	$O\left(W_{2D} \left(\kappa_1 \sqrt{P_z} + \frac{1-\kappa_1}{P_z^{3/6}}\right) + W_{2D} \left(\kappa_1 \frac{P_z \log P_z}{\sqrt{P}}\right)\right)^\dagger$
Latency	$O(n)$	$O\left(L_{2D} \left(\kappa_2 n^{-1/3} + \frac{1}{P_z}\right)\right)^\ddagger$

[†] The first and second term in the expression for W_{3D} represent W_{3D}^{xy} and W_{3D}^z , respectively.

$\kappa_1 = \frac{2(\alpha-1)}{\alpha^4-1}$, where $\alpha = 2^{1/3}$.

[‡] $\kappa_2 = \frac{\alpha}{\alpha+1}$

In both tables, the formulae of communication volumes in the 3D sparse LU are non-trivial functions depending on P_z . To provide visual perspective of the two functions, we plot these formula with increasing P_z while fixing the matrix dimension n to be 1 million and total processes P to be 1024, see fig. 5. In both cases, there are some sweet spots for the P_z values that would minimize the communication volume. It is not that the larger P_z the less communication. Note that these formula are just in “big O” sense and do not contain the constant prefactors in each case. Therefore, the relative communication volumes between the 3D and 2D SpLU are just notional magnitudes and should not be considered the actual differences on a real machine. In practice, the Sao’s paper showed that P_z ranging from 8 – 16 are usually good for both planer and non-planer problems.

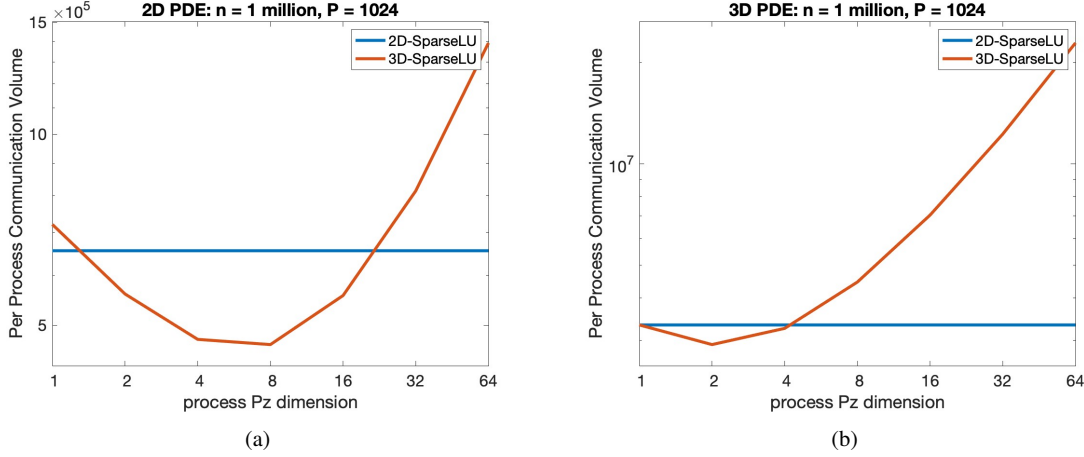


Fig. 5: Asymptotic per process communication volumes given in Tables 1 and 2, with different P_z settings.

Communication-avoiding 3D SpTRSV

Following the same 3D SpLU principle and the distribution of the LU factors, Sao et al. designed a new 3D SpTRSV algorithm [188]. Since the diagonal block of the L (or U) factor associated with a leaf subtree or an ancestor separator resides only on 2D process grids, the corresponding SpTRSV operations would still use the prior 2D algorithm. Similar to the 3D LU, there is some replicated data (memory) on the common ancestor nodes at the higher levels of etree. Different from LU, where the replicated memory is for the partial Schur complement updates, here in SpTRSV, the replicated memory is for the partial updates to the right-hand side matrix B . Each 2D grid accumulates the local partial sum that is reduced to update a part of right-hand side corresponding to the common ancestor nodes.

Consider solving the lower triangular system $LY = B$, using the same example in fig. 4. The right-hand side B (and solution Y) is partitioned into 7 block rows, $B_0, B_1, \dots, B_6$, corresponding to the 7 etree nodes. let S_i^j denote the partial update that is computed and residing in the process Grid- j but is destined to the etree node i for block B_i . At the leaf level, Grid-0 and Grid-1 first solve nodes 3 and 4 in parallel, obtaining $Y_3 = L_3^{-1} B_3$ and $Y_4 = L_4^{-1} B_4$. Grid-0 produces the partial updates $S_1^0 = -L_{1,3} \cdot Y_3$ and $S_0^0 = -L_{0,3} \cdot Y_3$ destined to B_1 and B_0 , respectively. Grid-1 produces the partial updates $S_1^1 = -L_{1,4} \cdot Y_4$ and $S_0^1 = -L_{0,4} \cdot Y_4$ destined to B_1 and B_0 . The two grids reduce the updates at nodes 1 and 0, forming the updated $B_1 = B_1 + S_1^0 + S_1^1$ and the pending update $S_0^0 = S_0^0 + S_0^1$ on Grid-0. Grid-0 proceeds to solve $Y_1 = L_1^{-1} \cdot B_1$ using the 2D algorithm. In parallel, similar operations occur for Grid-2 and Grid-3 along the etree nodes 5, 6, 2 and 0. At one level up, only Grid-0 and Grid-2 are active. They solve for $Y_1 = L_1^{-1} \cdot B_1$ and $Y_2 = L_2^{-1} \cdot B_2$. They then reduce the merged updates $S_0^0 - L_{0,1} \cdot Y_1$ (Grid-0) and $S_2^0 - L_{0,2} \cdot Y_2$ (Grid-2) on node 0 to Grid-0. At the root of the etree, only Grid-0 solves for Y_0 using the 2D algorithm. The generalization of this process for any $P_z = 2^l$ is given in [188, Algorithm 2]. Liu et al.[147] further improved the 3D SpTRSV algorithm by allowing replicated computation on each grid by setting the corresponding RHS to 0. This greatly simplifies the 3D algorithm design, and removes the need for inter-grid communication and synchronization at each level, albeit with moderately increased inner-grid communication and computation. This approach achieved up to 3.45 $\times$ additional speedups compared with the above-mentioned algorithm [188] on up to 2048 CPU cores.

2.3 Communication lower bounds

Even though the 3D CA algorithm framework implemented in SuperLU_DIST has low communication complexity among the widely used parallel sparse direct solvers, a natural question arises whether this communication complexity reaches the lower bound, that is, whether this is a communication optimal algorithm and whether we can continue to seek better algorithms to close the gap from optimum. The answer to this question is not simple. Firstly, the communication cost consists of two parts: bandwidth (volume) and latency (number of messages). Some algorithms may minimize

Table 3: Asymptotic communication cost and volume for 2D SpTRSV and 3D SpTRSV, on planar (2D PDE) and non-planar (3D-PDE) input problems

Problem type	Communication	2D SpTRSV	3D SpTRSV
Planar (2D PDE)	Cost (W)	$O\left(\frac{n}{\sqrt{P}} + \sqrt{n}\right)$	$O\left(\frac{n}{\sqrt{P_z P}} + \sqrt{n}\right)$
	Average Volume	$O\left(\frac{n}{\sqrt{P}}\right)$	$O\left(\frac{n}{\sqrt{P_z P}}\right)$
	Max Volume	$O\left(\frac{n}{\sqrt{P}}\right)$	$O\left(\frac{n}{\sqrt{P_z P}} + \frac{\sqrt{n P_z}}{\sqrt{P}}\right)$
Non-Planar (3D PDE)	Cost (W)	$O\left(\frac{n}{\sqrt{P}} + n^{2/3}\right)$	$O\left(\frac{n}{\sqrt{P_z P}} + n^{2/3}\right)$
	Average Volume	$O\left(\frac{n}{\sqrt{P}}\right)$	$O\left(\frac{n}{\sqrt{P_z P}}\right)$
	Max Volume	$O\left(\frac{n}{\sqrt{P}}\right)$	$O\left(\frac{n}{\sqrt{P_z P}} + n^{2/3} \frac{\sqrt{P_z}}{\sqrt{P}}\right)$

one cost but not both, or may be able to trade-off one for another. Secondly, some algorithms may be able to trade-off floating point operations and/or memory for lower communication cost. Thirdly, for sparse matrices, the optimality of an algorithm may depend on the sparsity pattern.

Let F denote the number of floating point operations per process, M denote the local memory size. For a broad class of matrix-multiplication-like linear algebra algorithms, dense or sparse, it was established that the lower bounds on communication bandwidth and communication latency are:

$$W = \Omega\left(\frac{F}{\sqrt{M}}\right), \quad S = \Omega\left(\frac{F}{M^{3/2}}\right) \quad (3)$$

see [27] and the historical notes therein. These formula suggest that when increasing the local memory size, the lower bounds for both W and S can be decreasing. This intuition prompted the development of some CA algorithms that replicate some data for reduced communication. In [196], Solomonik and Demmel presented a 2.5D dense LU algorithm that arranges P processes as c layers of 2D process grids. The dense matrix is duplicated on each layer, thus, the algorithm uses $M \approx cn^2/P$ local memory. The algorithm reduces the bandwidth by a factor of $\sqrt{c}$ in comparison with standard 2D LU algorithms (when $c = 1$). However, it sends a factor of $\sqrt{c}$ more messages than the 2D algorithm.

For SpLU, the two available open-source implementations with low communication complexity are: the multifrontal approach with subtree-to-subcube mapping proposed by Gupta et al. [98] and the right-looking 3D mapping approach proposed by Sao et al. (see section 2.2 and [192]). Both of these approaches fully exploit the etree parallelism and do not suffer the latency cost increase as does the 2.5D dense LU. Gupta's algorithm partitions the 2D process grid into smaller 2D process grids that are assigned to the frontal matrices at different levels. Sao's 3D mapping approach partitions the 3D process grid into smaller 3D process grids. The essence of both approach is to use multiple copies of the matrix to perform multiple Schur-complement updates in parallel. The low communication complexity of the multifrontal algorithm is also attributed to trading additional memory for reduced communication. This additional memory stores the multiple partial update matrices, i.e., the T matrices in fig. 1. For three-dimensional and higher-dimensional sparse matrices, the sparse LU operation is dominated by the dense LU at the top level separator, therefore, the behavior of the both Gupta's and Sao's algorithms are optimal in terms of communication. But, for sparse matrices associated with the planar graphs, both algorithms do not achieve the communication lower bound specified by eq. (3). Consider a model planar problem that comes from finite difference discretization of 2D grid of size $n = k \times k$, Gupta's algorithm has a gap of $\sqrt{\log n}$ from bandwidth lower bound, and Sao's algorithm has a gap of $\log n$ (see last column of table 1, lower bound is $F/\sqrt{M} = n/(\sqrt{P}\sqrt{\log n})$).

In [189], Sao et al. proposed a nearly-optimal algorithm in terms of communication, which combines the subtree-to-subcube mapping and 3D mapping approach. Here, the lower level subtree of the etree uses subtree-to-subcube mapping and the higher levels subtree use the 3D mapping algorithm. Thus, for higher level subtrees, data replication is used up to a factor of $\log n$, without increasing the asymptotic memory cost. This hybrid algorithm achieves communication lower bound within a factor of $\log \log n$, a considerable improvement over the $\sqrt{\log n}$ and $\log n$ factors of existing approaches.

However, it remains an open problem to implement this last communication-optimal algorithm.

The software details of the 3D sparse direct solver SuperLU_DIST are described in [135].

3 Uncovering Fine-grain Parallelism for GPU Acceleration

In the exascale computing era GPUs have become mainstream computing engines. In the November 2025 edition of the TOP500 supercomputer list, all four exascale machines are using various generations of GPUs from AMD, Intel and NVIDIA.⁶ On those machines, more than 90% of the computing power comes from GPUs. Despite their increasing adoption in scientific computing, GPU architectures present unique challenges for sparse computations. Modern GPUs can have thousands of cores, enabling them to run a large number of threads concurrently. This high thread count allows for simultaneous execution of many operations, greatly increasing throughput. This type of SIMD data-parallel model is well-suited to large dense matrix operation. However, for sparse matrix operations, task parallelism is often more prevalent than data parallelism. Significant efforts have been made to expose fine-grained data parallelism.

Programming on GPUs often requires explicit management of threads and thread blocks. Developers need to understand how to organize computations into many small kernels and manage synchronization between threads, which can be non-intuitive. While GPUs can handle many threads in parallel, their architectures are not designed for handling complex control flows. Therefore, most GPU-capable sparse direct solvers still use CPUs to handle control flows and use GPUs only in the *offload mode*.

About fifteen years ago, several CPU-GPU SpLU algorithms were developed that mostly use GPU as a BLAS accelerator (i.e., dense LU, TRSM and GEMM in fig. 1) [156, 226, 84, 203]. All of these earlier developments focused on single GPU or a few GPUs on a single node. Starting 2014, Sao et al. initiated the first line of work on large scale distributed memory GPU accelerated sparse direct solvers with implementations in SuperLU_DIST. This required using the MPI+OpenMP+CUDA programming model with delicate coordinations of both inter-node communication, intra-node CPU-GPU communication and intra-node multithreading on each multicore node. In their first distributed memory GPU-enabled SpLU algorithm [191], the only part offloaded to GPUs is the GEMM computation that makes up the contribution to each Schur complement update. They leveraged NVIDIA's cuBLAS library for the GEMM calls on the GPU side. To overcome the cost of data transfer between the CPU and GPU memories, they developed a sophisticated software pipelining for overlapping the GEMM operations on GPU with the Scatter operations on CPU. This requires creating multiple CUDA streams to split a single large GEMM or Scatter operation into smaller pieces. This algorithm achieved up to $2\times$ speed up on 8 nodes with 16 Tesla "Fermi" GPUs. However, for truly sparse matrices, the performance gain is marginal. Their second algorithm [190] also offloaded the Schur complement update to GPU, including both GEMM and Scatter operations. The algorithm is called HALO for Highly Asynchronous Lazy Offload, referring to combining asynchronous use of both CPU and GPU for each Schur complement update accounting for the GPU memory size and the Schur complement matrix size. The asynchronous and lazy updates serve to hide and reduce communication over PCIe link between CPU and GPU and to overcome the small GPU memory capacity. This strategy delivered an additional $2.5\times$ speedup over the GEMM-only-offload approach. In their subsequent effort [192], on top of the HALO strategy, Sao et al. added additional GPU offloading for the block diagonal dense LU and the L and U panel factorizations, leveraging the cuBLAS TRSM routine and the cuSparse GETRF routine.

In [87], Ghysels and Synk developed the first distributed memory GPU accelerated symmetric-pattern multifrontal sparse direct solver with implementation in STRUMPACK. Their algorithm offloads both dense matrix operations and the sparse scatter-gather operations to GPUs. They leveraged various vendor libraries for the dense operations, such as NVIDIA's cuBLAS and cuSOLVER and AMD's rocBLAS and rocSOLVE. For the smaller frontal matrices they developed custom CUDA and HIP kernels to reduce kernel launch overhead. The multi-GPU setting uses SLATE [81] as a modern GPU-aware replacement for ScaLAPACK. On 4 nodes of the Summit machine at Oak Ridge National Laboratory, the code is $10\times$ faster when using all 24 V100 GPUs compared to when it only uses the 168 POWER9 CPU cores. On 8 Summit nodes, using 48 V100 GPUs, the sparse solver reaches over 50TFlop/s.

In a more recent effort in PanguLU [80], Fu et al. developed a different SpLU algorithm which does not use any dense supernode data structure; instead, they use a uniform 2D block partition of the sparse matrix, in which each block is generally sparse and represented in the standard Compressed Sparse Column (CSC) format. In order to accommodate a vast number of different sparsity patterns, they developed 17 different sparse kernels for block diagonal LU, panel lower and upper triangular solutions and Schur complement update. Some of them are for CPU and some others for GPU. Based on the extensive microbenchmarks of various block patterns arising from different sparse matrices, they derived four decision trees to choose a specific kernel dynamically based on each block's sparsity pattern. Accompanied with an efficient asynchronous scheduling strategy for parallel block operations, PanguLU achieved strong scaling of $47\times$ and $74\times$ on 128 NVIDIA A100 GPUs and 128 AMD MPI50 GPUs, respectively, compared to single GPUs.

⁶ <https://www.top500.org/lists/top500/2025/11/>

On the new generation of GPUs, the memory size is much larger than earlier generations. For example, an NVIDIA GH200 (2024) Grace Hopper GPU has nearly 100 GB memory whereas an NVIDIA Tesla M2090 (2011) GPU had only 6 GB memory. As such, many applications were ported to run entirely on GPU and there is a demand for the algebraic solvers to run entirely on the GPUs. The recent work of Caracal SpLU algorithm [186] epitomizes the "GPU-resident" paradigm. Two main techniques developed by Ren et al. greatly enhanced the single GPU utilization: 1) a fine-grained static scheduling algorithm that utilize multiple GPU streams to perform TRSM and GEMM on multiple sub-blocks simultaneously, and 2) a memory caching algorithm and clever use of GPU memory pool to enhance data reuse and reduce the frequency of calling `cudaMalloc/cudaFree` that may cause GPU memory fragmentation. On a single NVIDIA A100 GPU, Caracal achieved up to 21% of the A100's theoretical peak performance, 7 $\times$ speedup over SuperLU_DIST, 94 $\times$ speedup over PanguLU, 16 $\times$ speedup over PaTiX. Even though the Caracal approach eliminates the need for CPU-GPU data transfers during the LU factorization, the inter-GPU communication is still initiated on the CPU. That is, CPUs are not entirely out of the picture. From their limited scaling study using only 4 GPUs, their multi-GPU results are not superior to the other methods. Further improvement is needed for efficient use of large number of GPUs.

Batching

In the same vein as the GPU-resident solvers that intend to reduce the number of kernel launches, *batching* also emerged as an efficient algorithm paradigm for GPUs. When executing many small tasks, the overhead of launching kernels and managing resources can dominate the computation time. Batching reduces this overhead by executing a large number of operations within a single kernel call, increasing overall throughput. This leads to significant speedups compared to executing each task sequentially. The batched algorithm is particularly suitable for applications that involve a large number of independent and *small to medium sized* linear algebra problems. These problems arise from combustion modeling, bio-chemical modeling, fusion plasma modeling, and nuclear physics simulations, to name a few. The dimension of each matrix is of order hundreds to tens of thousands. Most of the GPU throughput would be wasted if we solve each of them individually. The well-known libraries supporting dense batched matrix operations include MAGMA [3, 1] and KBLAS [114]. The newer library Kokkos Kernels supports batched linear algebra operations for both dense and sparse matrices [119].

The batched dense operations are also critical components to enable development of fast sparse direct solvers on GPUs. For example, in the GPU version of STRUMPACK, each level of the assembly tree involves multiple independent diagonal LU, off-diagonal TRSM and GEMM for the update matrices. These are all dense operations and can be organized as three batched routines across all the nodes of one level of the tree. However, the sub-matrices within a level are not of the same size. Abdelfattah et al. expanded the batched MAGMA library to handle irregular patterns of matrices in a batch and added new routines `irrLU`, `irrTRSM` and `irrGEMM` [2]. They showed that the new MAGMA enabled over 6 $\times$ speedup for STRUMPACK running on a single AMD MI100 GPU.

The SuperLU team led the development of the batched sparse direct solver interface and the implementation in SuperLU_DIST [42]. A salient feature of the new interface is to allow heterogeneous batch structure in that each linear system can have different size, sparsity pattern and numerical property. The interface allows each system in the batch to have different preprocessing: equilibration, row and column reordering, depending on the numerical values and sparsity pattern of each individual system. The results with some combustion and fusion applications demonstrate that the batched sparse direct solver can be order of magnitude faster than the batched banded solver or the batched dense solver in MAGMA.

There are several prominent efforts in developing batching sparse iterative solvers and preconditioners. The modern GPU-oriented Ginkgo library [23] supports batched iterative solvers for both symmetric and nonsymmetric sparse linear systems. Current version supports a fixed sparsity pattern across all the matrices in the batch. That is, their BatchCsr or BatchEil formats stores only one copy of the meta-graph data structure, with different numerical values [113]. In addition, Ginkgo supports batched Block-Jacobi preconditioner for which the inversion of the diagonal block is performed via a batched variable size Gauss-Jordan elimination. The Kokkos Kernels library also supports a number of batched iterative solvers including CG and GMRES. Liegeois et al. described in detail how they optimized the batched sparse matrix vector product (SpMV) which is the performance-dominant operation in most sparse iterative solvers [138].

4 Acceleration via Data-sparse Compression

Large-scale numerical linear algebra (NLA) algorithms, even possibly exploring exact structural sparsity and communication-avoiding techniques, can remain computationally expensive in terms of flop counts, storage units, communication and data movement. Moreover, modern HPC architectures are increasingly heterogeneous and compute-intensive, but the relatively slower improvement on memory bandwidth, communication latency and energy efficiency is still limiting the size of problems that can be solved. Data-sparse compression, as a competitive complement, refers to compact representations where the effective amount of information is much smaller than the ambient size of a dense matrix/tensor/array. Data-sparse compression reduces the amount of data that must be stored, moved and processed while maintaining a controllable accuracy level, and has emerged as a fundamental enabler in NLA and scientific computing.

The data-sparse compression can be applied to the entire dense matrix of interest, its judiciously selected dense submatrices, or dense submatrices of a sparse matrix, leading to significant reduction in the asymptotic complexity and/or its large constant prefactors of the original exact NLA algorithm. Data-sparse compression can be categorized into two complementary families of techniques, i.e., low-rank compression and lossy floating-point compression, which are summarized as follows:

Low-rank compression exploits mathematical or algebraic structure in matrices to approximate them efficiently and accurately. Many matrices arising from PDEs [71, 53, 73, 75], integral equations [30, 100], kernel methods in machine learning [185, 55, 58, 57, 11, 143, 169, 9] and optimization [78, 6, 8] are not arbitrary dense matrices: they exhibit latent separability, long-range separability, smoothness, or decay, which implies that submatrices of the matrix have rapidly decaying singular values and can be approximated by products of small matrices of rank far less than the dimension of the matrix. Low-rank compression become essential in direct solvers and factorization-based preconditioners (compressing dense frontal matrices and Schur complements), iterative solvers (compressing operator evaluations and Krylov bases), fast transforms, and inverse problems (accelerating Jacobian/Hessian actions and PDE-constrained optimization). In the context of direct solution of large-scale sparse and dense linear systems, low-rank compression includes single low-rank compression [213, 109], rank-structured matrix algorithms leveraging single-level low-rank compression [194, 5] or multilevel compression such as hierarchical matrices [208, 54, 124, 63, 170, 9], and their higher-order tensor extensions [152, 131]. These methods can effectively reduce the $O(n^3)$ computational complexity for factorizing a dense $n \times n$ matrix (or lower but still superlinear complexities for sparse matrices) to quasi-linear, linear or even sublinear complexities. Moreover, many modern low-rank compression methods can reduce the constant pre-factors, fully utilize parallel computers, and maintain low asymptotic complexity, making them capable of solving record-breaking large-scale dense and sparse linear systems.

Lossy floating-point compression exploits bit-level redundancy in numerical arrays. Rather than exploiting algebraic structure by low-rank compression, it reduces cost by quantizing or transforming data to a compressed representation with controlled errors. Matrices and multi-dimensional arrays, arising from PDE solutions and operators, machine learning features, fast transforms (e.g. FFT), and numerical algorithms (e.g., AMG), can exhibit local smoothness/correlation, and small dynamic range or low entropy, whose bit-level representation can be efficiently transformed or quantized to exploit the redundancy. Existing lossy compressors such as ZFP [140], SZ [65], and FPZIP [141] can significantly reduce the storage cost of large scientific data and leverage shared-memory and GPU parallelism. To the best of our knowledge, lossy floating-point compression has been mostly used in reducing the memory requirement or communication volume, albeit incurring manageable computational overheads during compressing and decompressing the data.

It is worth emphasizing that low-rank compression and lossy floating-point compression are two complimentary families of data-sparse compression algorithms: the former exploits mathematical structure (long-range smoothness and spectral decay) and the latter exploits bit-level redundancy (local smoothness and correlation). Oftentimes one can attain optimal compression by hybridizing the two algorithms such as [122, 60, 176, 118] in practice and we will see a few such examples in section 5.

In the rest of this section, we will focus on the description of the fundamentals of rank-structured matrices for the direct solution of dense matrices.

4.1 Overview of rank-structured matrix algorithms

Large-scale dense linear systems $AX = B$ arise from many computational science and engineering, machine learning and statistics problems, including matrices in boundary element methods and volume integral equations, Fourier integral operators, fractional Laplacian operators, kernel matrices and covariance matrices in machine learning, Hessian matrices in PDE-constrained optimization, and effectively dense matrices due to fill-in of a sparse direct factorization for PDEs. Exact arithmetics-based solvers for these large linear systems can be computationally very demanding even leveraging modern parallel HPC architectures. This is particularly the case when the system becomes ill-conditioned and/or involves many right-hand side vectors B , as in multi-physics, multi-scale simulations and machine learning pipelines. That said, these matrices often exhibit rich mathematical and physical structures that can be exploited to significantly reduce the solver complexities. We will see a few such examples in Section 4.2.

Rank-structured matrix algorithms [164] aim at solving these large $n \times n$ linear systems with less than the conventional methods' $O(n^2)$ memory and $O(n^3)$ computational requirements. Taking hierarchical matrices as an example, they leverage the ideas of multilevel matrix partitioning and low-rank compression. They can be viewed as the algebraic version of the fast multipole methods with additional support for matrix inversion. After a proper *preprocessing* phase that reorders the matrix rows/columns and hierarchically clusters them into groups, certain off-diagonal blocks at each level l of the reordered matrix reveal geometric structure or smooth interactions, leading to rapidly decaying singular values. This means these blocks can be approximated, up to a user-specified accuracy tolerance ϵ , as low-rank matrices:

$$A_{st}^l \approx U_{st} V_{st}^T. \quad (4)$$

Here A_{st}^l is the $n_s \times n_t$ block corresponding to row group s and column group t , U_{st} and V_{st}^T are the low-rank factors, and the rank $r \ll \min(n_s, n_t)$ and depends on the specified tolerance ϵ and the nature of the underlying kernel. As a result, one needs far less than $O(n_s n_t)$ flop operations and storage units for this block. After compressing all off-diagonal blocks at each level of the hierarchical clustering (which is referred to as the *construction* phase), one ends up with a compressed representation $H \approx A$ with almost $O(nr)$ memory requirement.

Once the construction of H is finished, many basic operations on A can be approximated and accelerated using H instead of A . These include *factorization* of H into H^{-1} [9, 92] or its equivalents (e.g., ULV [54, 52], LU [92, 194, 200], QR [120], Cholesky [214, 92] and telescoping factorization [166, 124]), *apply* of H or H^{-1} to given vectors (also called the *solve* phase), recompression of the composition of H and other operators into a new rank-structured matrix, etc.

There are a wide variety of rank-structured matrices invented in the applied mathematics, computer science and engineering (e.g., acoustics, seismics, electromagnetics and fluid dynamics) societies. Multilevel partitioning-based algorithms, i.e., hierarchical matrices, can be roughly classified into four categories depending on two criteria, i.e., the admissibility condition and the shared basis property. The admissibility condition refers to whether the low-rank compression of (4) is applied to all off-diagonal blocks (dubbed weak admissibility) or only those are “well-separated” from each other using certain distance metrics (dubbed strong admissibility); the shared basis property refers to whether the U_{st} (and V_{st}) factors for two row (and column) groups in (4) share common column (and row) bases with other groups. Weak-admissible and independent-basis algorithms typically refer to hierarchically off-diagonal low-rank (HODLR) matrices [9], and strong-admissible and independent-basis algorithms include $\mathcal{H}$ matrices [208] and its variants [117]. Weak-admissible and shared-basis algorithms include hierarchically semi-separable (HSS) matrices [54, 214, 212, 90], hierarchical interpolative factorization (HIF) [124], and strong-admissible and shared-basis algorithms include $\mathcal{H}^2$ matrices [208, 157], inverse fast multipole methods (IFMM) [63], and strong recursive skeletonization factorization (RS-S) [170], etc. In general, strong admissibility, as opposed to weak admissibility, leads to smaller ranks for high-dimensional problems (e.g., solving PDEs on 3D manifolds or in 3D spaces, or performing kernel regression in high-dimensional feature spaces), but results in higher computational costs for low-dimensional problems; shared-basis based algorithms are more sophisticated but computationally less expensive than independent-basis based algorithms. A summary of the four major classes of hierarchical matrix algorithms is provided in Table 4.

When dealing with matrices arising from oscillatory problems such as high-frequency wave equations and oscillatory integral transforms, the rank r in eq. (4) can still remain high even for strong-admissibility-based hierarchical matrix algorithms. In this regime, one can replace the low-rank compression A_{st}^l in eq. (4) with multilevel compression algorithms such as directional fast multipole method [70] or butterfly compression [168, 155], which leads to efficient extensions of hierarchical matrices capable of solving oscillatory problems with quasi-linear computational complexity. These algorithms include strong-admissible formats e.g., directional $\mathcal{H}^2$ matrices [47] and hierarchical block LU-

based solvers [96, 102], as well as weak-admissible formats e.g., hierarchical off-diagonal butterfly matrices (HODBF) [149, 193] and directional preconditioners [225].

As opposed to the hierarchical matrices, single-level partitioning based algorithms, named block low-rank (BLR) matrices or tile low-rank matrices [194, 5], only compress off-diagonal blocks of A_{st}^l in eq. (4) at a fixed level l . These algorithms are simpler than hierarchical matrices but can also provide significant computational and memory savings. Just like hierarchical matrices, BLR algorithms also have variants that use shared bases or different admissibility conditions [182, 24, 16].

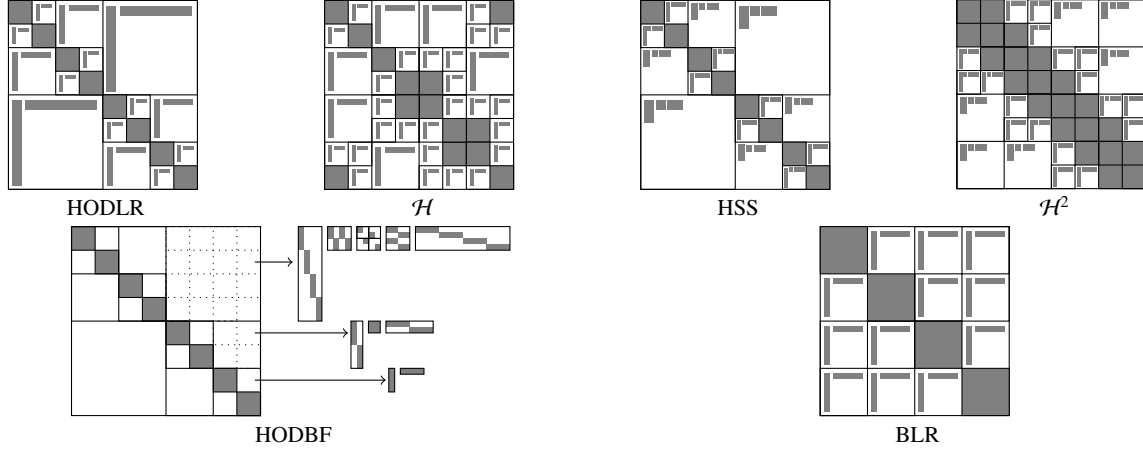


Fig. 6: Illustration of several types of hierarchical matrices (HODLR, $\mathcal{H}$, HSS and $\mathcal{H}^2$, HODBF (the butterfly extension of HODLR), and BLR (single-level partitioning-based)).

Format	Storage	Construction	Factorization	Apply	Admissibility	Shared Basis	Dimensionality
HODLR	$O(nr \log n)$	$O(nr^\alpha \log^\beta n)$	$O(nr^2 \log^2 n)$	$O(nr \log n)$	Weak	No	$d \leq 2$
$\mathcal{H}$	$O(nr \log n)$	$O(nr^\alpha \log^\beta n)$	$O(nr^2 \log^2 n)$	$O(nr \log n)$	Strong	No	$d \geq 3$
HSS	$O(nr)$	$O(nr^\alpha)$	$O(nr^2)$	$O(nr)$	Weak	Yes	$d \leq 2$
$\mathcal{H}^2$	$O(nr)$	$O(nr^\alpha)$	$O(nr^2)$	$O(nr)$	Strong	Yes	$d \geq 3$

Table 4: Summary of four major classes of hierarchical matrix representation of a $n \times n$ matrix. r denotes the maximum numerical rank in the off-diagonal blocks, d denotes the dimensionality of the underlying scientific or machine learning kernels. $\alpha = 1, 2$, $\beta = 1, 2$. “Apply” represents application of the matrix or its inverse to a single vector. Note that $\mathcal{H}$ and $\mathcal{H}^2$ matrices also support weak admissibility, but their major advantage over other hierarchical matrix algorithms is attributed to their support for strong admissibility for high-dimensional problems.

4.2 Existence of hierarchical matrix structure in practical applications

The efficiency of the rank-structured matrix algorithms heavily relies on, after certain preprocessing, the existence of the low-rank structure for the matrix operators appearing in many scientific computing, physics, engineering, and machine learning applications. Here we list a few application examples where rank-structured matrix algorithms are known to be effective.

4.2.1 Integral equations

For the simulation of wave (e.g., acoustics, elastics, electromagnetics and optics [161, 200, 194, 193, 166, 61, 100, 94, 151]), fluid (e.g. Stokes flow [29]) and plasma (e.g., MHD [116], Poisson solvers [37]) phenomena, one oftentimes solves a linear system with A in the form of

$$A_{ij} = \int_D \psi_i(y) G(x, y) \phi_j(y) dy. \quad (5)$$

Here ψ_i and ϕ_j are local basis functions (oftentimes $\psi_i = \phi_i$), D is the domain of integration, e.g., some curve or volume, and G is the Green's function for Helmholtz, Laplace, Yukawa or Stokes kernels, etc. For example, the Green's function of the integral equations (IE) for the 3D Laplace kernel is

$$G(x, y) = \frac{1}{4\pi|x - y|}. \quad (6)$$

For off-diagonal blocks representing Green's function interaction between well-separated spatial groups, one can prove the algebraic compressibility for analytical [224, 71] and semi-analytical [154] Green's function for both smooth and oscillatory kernels. On the other hand, the compressibility of off-diagonal blocks of A^{-1} , essential for preserving the computational advantage of fast direct solvers, remains an active research area [22, 74, 72].

4.2.2 Kernel matrices

Many machine learning and statistics algorithms, e.g., kernel ridge regression and Gaussian process, require the inversion of kernel matrices in the form of a Gaussian kernel

$$A_{ij} = \exp - \frac{\|x_i - x_j\|^2}{2\sigma^2} \quad (7)$$

with hyperparameter σ , or many other kernels. Due to smoothness or rapid decay (with respect to $\|x_i - x_j\|$) of the kernels, the kernel matrix A exhibits low numerical ranks in the off-diagonal blocks and rank-structured matrix algorithms have been widely used [185, 55, 58, 57, 11, 143, 169, 9]. Moreover, when optimizing hyperparameters in Gaussian process with gradient-based methods, one often constructs a hierarchical matrix representation for $\partial_\sigma A$ as well [82].

4.2.3 PDEs and Schur Complements

Discretization of many elliptic PDEs (e.g., Poisson, Helmholtz, Yukawa, convection-diffusion, convection-reaction, and incompressible Stokes, etc.) leads to a sparse linear system A . In supernodal or multifrontal sparse direct solvers, block elimination of the properly permuted system leads to Schur complements in the form of

$$S = A_{ss} - A_{sr} A_{rr}^{-1} A_{rs}, \quad A = \begin{bmatrix} A_{rr} & A_{rs} \\ A_{sr} & A_{ss} \end{bmatrix} \quad (8)$$

. The Schur complement can be viewed as the numerical Green's function for the underlying PDE [71, 53, 73, 75] and hence exhibits low-rank structures. Here we assume A is 2×2 block partitioned for simplicity. Note that in reality, the block elimination process often leads to Schur complements in the form of

$$S = F_{ss} - F_{sr} F_{rr}^{-1} F_{rs}. \quad (9)$$

Here F are the dense frontal matrices where F_{rr} and F_{ss} are represented as hierarchical matrices, and F_{rs} and F_{sr} are presented as either hierarchical matrices or single low-rank compression [60], depending on the admissibility condition.

4.2.4 PDE-constrained inverse problem and optimization

In PDE-constrained optimization and inverse problems [78, 6, 8], one optimizes some misfit term $f(u(m))$ with respect to model parameter m subject to a PDE constraint $A(m)u - b = 0$, this leads to a Hessian matrix in the form of

$$G = A^{-T} \partial_{mm}^2 f A^{-1} + S. \quad (10)$$

Here A is the discretized PDE operator, A^{-1} can be computed via hierarchical matrix algorithms, $\partial_{mm}^2 f$ and S are local operators. One can show that G also inherits the hierarchical matrix structure of A^{-1} [92, 8, 32] and can be efficiently inverted by hierarchical matrix algorithms.

In addition to the above-mentioned classes of matrices, many other special matrices such as Toeplitz matrix, Hankel matrix, Fourier integral operator, and spherical harmonic transform also exhibit hierarchical rank structures.

4.3 Algorithm description with a $\mathcal{H}^2$ matrix example

4.3.1 Preprocessing

The preprocessing phase is typically required for all existing rank-structured matrix algorithms to reveal the proper hierarchical structure to achieve optimal compressibility and computational efficiency. The preprocessing phase determines (a) clustering: permutation information that block partitions the matrix into, typically, multiple levels and (b) admissibility assignment: which blocks at each level of the permuted matrix can be algebraically compressed. When geometrical information is available for each row/column, e.g., Cartesian coordinates x_i in (7), or any geometrical point in the support of local basis functions ψ_i, ϕ_j in (5), clustering algorithms such as KD tree, cobblestone-like [194] or random projection tree [55] are commonly used. For dense frontal matrices arising from sparse multifrontal solvers with no geometrical information, the frontal matrices are clustered using graph partitioning algorithms such as recursive bisection [148, 86].

Similarly, admissibility assignment needs to estimate how far away a pair of block rows and block columns are from each other. When geometrical information is available, the distance and diameters of two geometrical groups can be easily computed via geometrical distance (Note that even frontal matrices for PDEs can inherit the geometrical information [231] when the PDE is discretized on e.g., a 3D Cartesian grid). However, for (SPD) kernel matrices algebraically constructed without geometrical information, kernel distances have been exploited to determine the admissibility [231]. Similarly, for frontal matrices without geometrical information, one can rely on graph distance [148], or adaptive algorithms (i.e., attempting compressing each block but further partitioning it if the rank is too high) [59]. In addition to admissibility, sometimes it suffices to compute a set of nearest neighbors for each row/column. This can be computed with k-nearest neighbor search [36], approximate nearest neighbor search [55] or graph-based algorithms [148].

As an example, consider a $\mathcal{H}^2$ representation of the kernel matrix in eq. (7) with geometrical information x_i . A KD tree-based clustering will generate a cluster tree I of $L = O(\log n)$ levels (see fig. 7(b)). Each pair of clusters (I_s, I_t) (or simply (s, t)) at each level l are deemed admissible

$$\text{adm}(s, t) = 1, \text{ if } \frac{\text{Diam}(s) + \text{Diam}(t)}{2} \leq \eta \text{Dist}(s, t) \quad (11)$$

where Diam and Dist are the diameters and distance of the two clusters. The block A_{st} for the admissible pair, i.e., admissible block, is algebraically compressed and the inadmissible block is kept as dense. Figure 7(a) shows the matrix structure with a set of $n = 2^{15}$ 3D geometry points using the admissibility parameter $\eta = 0.7$ (green and red indicate admissible and inadmissible blocks, respectively). The set of clusters that form inadmissible pairs with a cluster t are denoted as $\mathcal{N}_t$. The set of clusters (a) that form admissible pairs with cluster t and (b) whose parents form inadmissible pairs with parent of t , are denoted as $\mathcal{F}_t$. In $\mathcal{H}^2$ matrix format, each admissible block is compressed as $A_{st} = U_s B_{st} V_t^T$ where B_{st} is a $r \times r$ coupling matrix, and U_s and V_t^T are column and row bases. The basis for leaf nodes in the cluster tree are stored explicitly, and the basis for an inner node t of the cluster tree is defined in terms of the basis of its children t_1 and t_2 using transfer matrices E_{t_1} and E_{t_2} , resulting in a nested and shared basis:

$$U_t = \begin{bmatrix} U_{t_1} & \\ & U_{t_2} \end{bmatrix} \begin{bmatrix} E_{t_1} \\ E_{t_2} \end{bmatrix} \quad (12)$$

See the $\mathcal{H}^2$ illustration in Fig. 6 where the leaf-level inadmissible blocks, coupling matrices, leaf-level basis matrices, and higher-level transfer matrices are shown.

In addition to $\mathcal{H}^2$ matrices, different choices of admissibility parameter ($\eta \geq 1$ (weak admissibility) or $\eta \leq 0.5$ (strong admissibility)) and shared basis property lead to all the other hierarchical matrix formats in Table 4 and Fig. 6. It's worth noting that in independent basis-based formats such as $\mathcal{H}$ or HODLR matrices, the low-rank compression of each admissible block becomes $A_{st} \approx U_{st} V_{st}^T$ of (4) instead of $A_{st} \approx U_s B_{st} V_t^T$ (again, see Fig. 6).

4.3.2 Construction

The construction phase essentially computes the low-rank compression $A_{st} \approx U_{st} V_{st}^T$ of (4) for all admissible blocks to form a computationally efficient approximation $H \approx A$. In order to achieve faster than $\mathcal{O}(n^2)$ computational complexity for large matrices, one needs to use alternative algorithms to truncated SVD or rank-revealing QR that forms a full $n \times n$ admissible block. Instead, optimal-complexity algorithms rely on the assumption of either (a) any entry of the block A_{st} can be efficiently computed on the fly, or (b) the block and its transpose can be rapidly applied to any random vector Ω as $A_{st}\Omega$ and $A_{st}^T\Omega$. The former leads to purely algebraic algorithms such as adaptive cross approximation (ACA) or Nyström method [30, 31, 228, 153, 213], and proxy point or Chebyshev interpolation-based methods [220, 218] requiring geometrical information. These algorithms have been extended for butterfly compression as well [49, 133, 177]. On the other hand, the latter leads to random sketching-type algorithms such as randomized SVD [137], QR [217, 79], UTV [165] and butterfly [155, 133] algorithms. These faster algorithms can achieve up to an optimal $\mathcal{O}(nr)$ or $\mathcal{O}(nr^2)$ computational complexity.

When it comes to efficient construction of rank structured matrix representations, the above assumptions become (a) entry-evaluation: any entry of A can be efficiently computed, (b) fully sketching: $A\Omega$ and $A^T\Omega$ can be rapidly computed, or (c) partially sketching: a hybrid of (a) and (b). In practical applications, entry-evaluation applies to integral equations (5) and kernel matrices (7), etc.; randomly sketching applies to frontal matrices (9) and Hessian matrices (10), etc; partially sketching is often applied to frontal matrices (when the update term $F_{sr} F_{rr}^{-1} F_{rs}$ in (9) is formed as a low-rank compression first).

In the context of $\mathcal{H}^2$ matrices, the adaptation of fast low-rank compression algorithms becomes more sophisticated due to the shared-basis property. In other words, one needs to compute a single low-rank basis U_s of A_{st} , $t \notin \mathcal{N}_s$ for each cluster s and a single low-rank basis V_t of A_{st} , $s \notin \mathcal{N}_t$ for each cluster t via entry evaluation or sketching. Fast entry evaluation-based $\mathcal{H}^2$ algorithms include proxy surface-based [108, 169, 166, 218], polynomial interpolation-based [100, 35, 227], and ACA-type [33, 229] algorithms. On the other hand, fast sketching-based $\mathcal{H}^2$ algorithms include fully sketching-based top-down algorithms [139, 127, 227], fully sketching-based bottom-up algorithm [223], and partially sketching-based bottom-up algorithm [44].

Here we give a brief description of the partially sketching-based bottom-up algorithm [44] using the example of Fig. 7(b). Note that admissible blocks only exist at the bottom two levels. Assuming A is real and symmetric for simplicity of explanation, the algorithm uses interpolative decomposition (ID) for admissible blocks such that $A_{st} \approx U_s B_{st} U_t^T$, and $B_{st} = A(\tilde{I}_s, \tilde{I}_t)$ for some skeleton sets $\tilde{I}_s \subseteq I_s$ and $\tilde{I}_t \subseteq I_t$. From the sketching data $Y = A\Omega$ with some random matrix $\Omega \in \mathbb{R}^{n \times d}$, $d > r$, the algorithm first computes $Y_s = Y(I_s, :) - \sum_{b \in \mathcal{N}_s} D_{sb} \Omega_b$ with $\Omega_b = \Omega(I_b, :)$ for each leaf-level cluster s . Note that D_{sb} denote leaf-level inadmissible blocks and are explicitly computed via entry evaluation. Y_s effectively sketches the column space of all admissible blocks at block row s (see fig. 7(c)), and U_s and $\tilde{I}_s$ (also denoted as I_{s_S}) can be computed from Y_s via ID. Note that A in fig. 7(c) has been reordered such that the redundant indices $I_{s_R} = I_s \setminus I_{s_S}$ appear before skeleton indices I_{s_S} for each cluster s . After applying a transformation associated with U_s , only the skeletonized row $A(\tilde{I}_s, :)$ and column $A(:, \tilde{I}_s)$ remain (see Fig. 7(d)(e)). At next level, one computes $Y_s = \begin{bmatrix} U_{s_1} Y_{s_1} \\ U_{s_2} Y_{s_2} \end{bmatrix} - \begin{bmatrix} \sum_{b \in \mathcal{F}_{s_1}} B_{s_1 b} U_b^T \Omega_b \\ \sum_{b \in \mathcal{F}_{s_2}} B_{s_2 b} U_b^T \Omega_b \end{bmatrix}$ for each level-2 cluster s with s_1, s_2 being its children. Note that the coupling matrices $B_{s_1 b}$ and $B_{s_2 b}$ are explicitly computed via entry-evaluation (see the blocks in red in Fig. 7(f)) and Ω_b and T_b for a level-1 cluster b have been scaled by U_b . Y_s effectively sketches the column space of all admissible blocks shown in Green in Fig. 7(h) from which transfer matrices E_{s_1} and E_{s_2} can be computed via ID. Finally, the coupling matrices B_{st} at level 2 are explicitly formed via entry evaluation. As can be seen, this algorithm leverages both sketching and entry evaluation, and can achieve the optimal $\mathcal{O}(nr^2)$ computational complexity. Numerical results in [44] reports up

to a $1000\times$ speedup over an existing GPU implementation of the top-down sketching-based algorithm [227] and a $660\times$ speedup over a CPU implementation of the algorithm [127] (see Fig. 8).

In addition to $\mathcal{H}^2$ matrices, fast construction algorithms have also been applied in many other rank-structured matrix algorithms, including entry-evaluation-based algorithms for $\mathcal{H}$ [92], HODLR [11], HODBF [193], HSS [55] and BLR [194], as well as sketching-based algorithms for $\mathcal{H}$, HODLR [139, 127], HSS [91, 162, 126] and uniform BLR [182], etc.

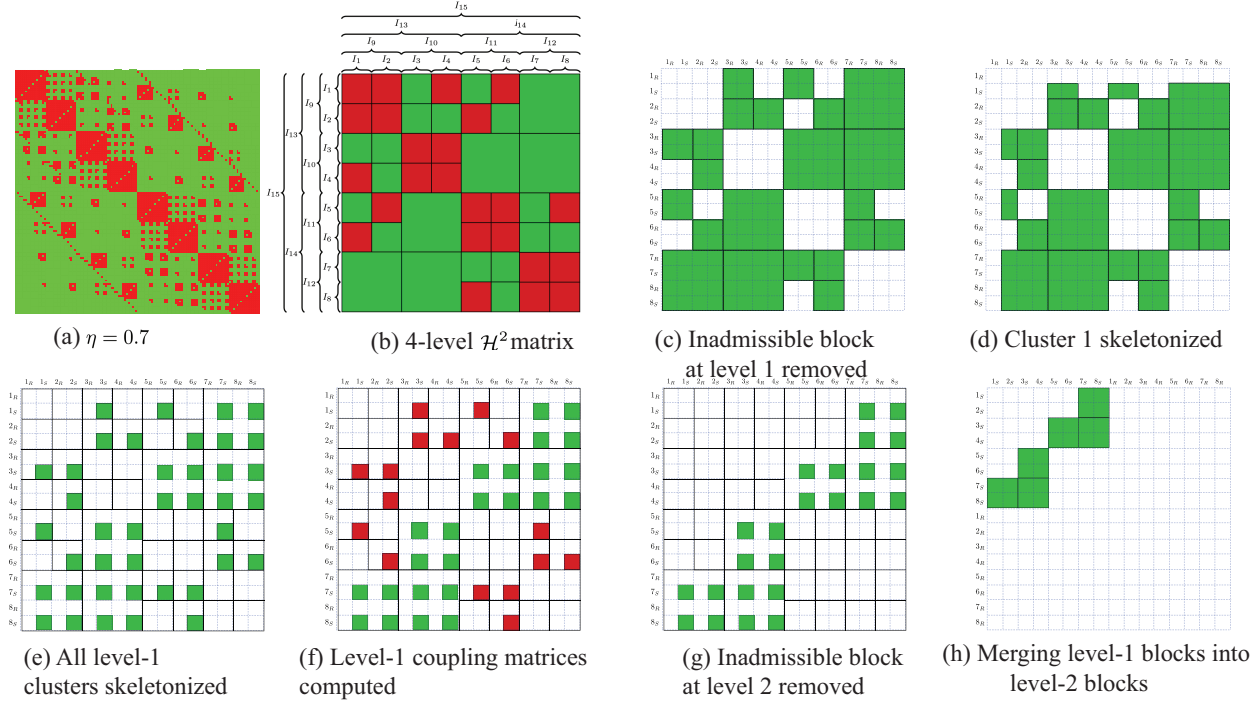


Fig. 7: [44, figs. 1 and 4] (a) Block partitioning of a hierarchical matrix for a 3D problem of size $N = 2^{15}$ with different η . (b) Illustration of a 4-level $\mathcal{H}^2$ matrix, (c)-(h) First two levels of the partially sketching-based algorithm in [44].

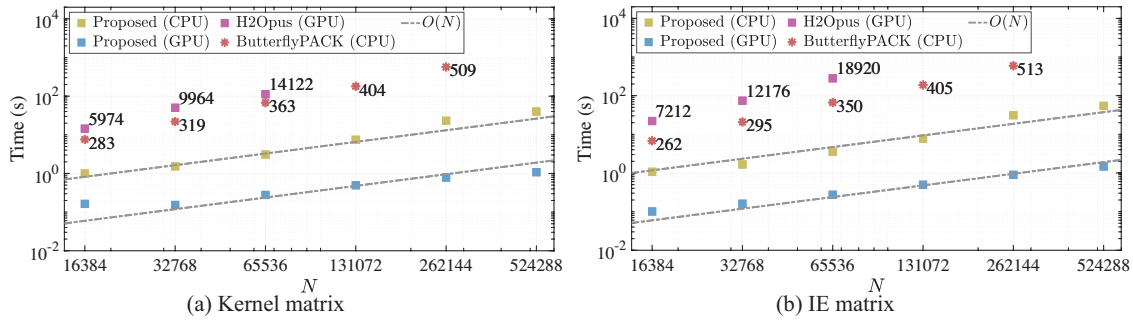


Fig. 8: [44, fig. 5] The time of the CPU and GPU implementations of the construction algorithm [44] for the kernel and IE matrices. Also shown is the top-down construction algorithm implemented in ButterflyPACK [127] and H2Opus [227], with its data points labeled with the total random samples d . In comparison, algorithm [44] requires only $d = 256$

4.3.3 Factorization and Solve

The factorization phase efficiently computes the inverse of the rank-structured matrix H^{-1} or its equivalent, by leveraging arithmetic operations in the compressed formats such as block addition and multiplication [92], Sherman-Morrison-Woodbury formula [9], and skeletonization [166]. Once H^{-1} has been computed, it can be used in the solve phase to compute $X = H^{-1}B$ for any RHS B leveraging fast arithmetic operations as well. It's worth noting that the factorization (and solve) algorithms can significantly differ from each other based on different rank-structured matrix formats. These include recursive block LU [39, 92, 96, 102] for $\mathcal{H}$ or $\mathcal{H}^2$ matrices, single-level block LU for BLR matrices [194], block ULV or Cholesky for HSS [54, 52, 214, 90], multiplicative inversion for HODLR [12, 11, 9] and HODBF [149, 193], recursive skeletonization for HSS and $\mathcal{H}^2$ [166, 170]. These algorithms can achieve an up to $O(nr^2)$ optimal computational complexity as shown in Table 4. While many factorization algorithms for weak-admissible formats such as HODLR [9] or HSS [166] can guarantee a constant growth factor in r during factorization, these formats are less preferable for high-dimensional problems as r does not stay independent of n due to weak admissibility. Although there exist specialized algorithms [61, 124] for PDEs on structured grids, strong-admissible formats such as $\mathcal{H}$ or $\mathcal{H}^2$ matrices exhibit much smaller ranks.

As efficient and parallel $\mathcal{H}^2$ factorization algorithms represent one state-of-the-art rank-structured matrix algorithm research area, we summarize existing $\mathcal{H}^2$ algorithms in the following. The algorithms of Börm et al. [40] and Ma et al. [157] represent the early $\mathcal{H}^2$ factorization algorithms based on recursive block addition/multiplication and local update, which can attain $O(nr^2)$ computational complexity, albeit with high constant prefactors. Later, inverse fast multipole (IFMM) methods [63, 10, 184, 52] were developed which rewrite the dense system as an extended sparse system using FMM bases and perform block LU elimination. More recently, the recursive skeletonization with strong admissibility (RS-S) [170] was invented leveraging both interpolative decomposition and block LU elimination. RS-S exhibits much smaller prefactors and is more amenable to efficient parallelization, hence there exist a sequence of follow-up works [136, 159, 158, 223, 200, 43].

Here we give a high-level overview of the $\mathcal{H}^2$ factorization algorithm in [43] using the 4-level $\mathcal{H}^2$ matrix of Fig. 7(b). For simplicity of explanation, we write $H = M + D + F$ where M and D contain respectively all admissible and inadmissible blocks at level 1 and F contains the fill-ins to be generated (see Fig. 9(a)). For cluster 1, let $Q_1 = \text{diag}(\tilde{Q}_1, I, \dots, I)$ where $\tilde{Q}_1 = [U_1^+, U_1]$, $\tilde{H} = Q_1^T H Q_1$ will skeletonize cluster 1. Next, LU elimination of $\tilde{H}_{1_R 1_R}$ by L_1 and U_1 will modify blocks in D (shown in blue in Fig. 9(b)) and introduce fill-ins in F (shown in black in Fig. 9(b)). Denote the matrix after LU elimination of s_R at level l by L_s and U_s to be A_s^l . We repeat these operations for cluster 2 with $A_2^1 = L_2 Q_2^T A_1^1 Q_2 U_2$ (see Fig. 9(c)) and for all the remaining level-1 clusters until $A_8^1 = L_8 Q_8^T A_7^1 Q_8 U_8$ (see Fig. 9(d)). Note that the fill-in blocks (in black) appearing at the row block of cluster s are used to augment the original U_s . The blocks in M and F corresponding to inadmissible blocks at level 2, marked in red in Fig. 9(e), are then moved into D . Finally, a permutation P^1 is applied to permute all the diagonal blocks of redundant indices to the lower right corner (see Fig. 9(f)), followed by a LU factorization $L_r^1 U_r^1$ that eliminates these redundant blocks. The above procedure is repeated for level 2, 3, $\dots$, L , until at the highest level L there is no more admissible blocks left and a final LU factorization $L_L U_L$ is performed. This LU-like factorization can be summarized as

$$H = \left(\prod_{l=1}^L \left(\prod_s Q_s L_s^{-1} \right) P^{l^T} L_r^l \right) L_L U_L \left(\prod_{l=L}^1 \left(U_r^l P^l \prod_s U_s^{-1} Q_s^T \right) \right). \quad (13)$$

It is worth noting that unlike weak-admissible formats such as HODLR and HSS, the rank growth of H^{-1} for $\mathcal{H}$ or $\mathcal{H}^2$ matrices are due to the fill-in blocks (in black in Fig. 9), which hasn't been fully understood to date [22, 74, 72].

Once the factorization in (13) has been computed, H^{-1} can be applied to any RHS vectors B by applying each factor in (13) one by one. Note that the inverse of Q_s , P^l , L_r^l and U_r^l are readily available now. As a few examples, Fig. 10 demonstrates the factorization time, factorization memory and solve time when applying the algorithm of [43] to 2D and 3D, IE and kernel matrices of varying sizes on a shared-memory CPU architecture. As can be clearly seen, the $O(n)$ complexities have been validated.

Needless to say, depending on the accuracy of U_s and condition number of A , the factorization in (13) can be either used as a direct solution operator or efficient preconditioner.

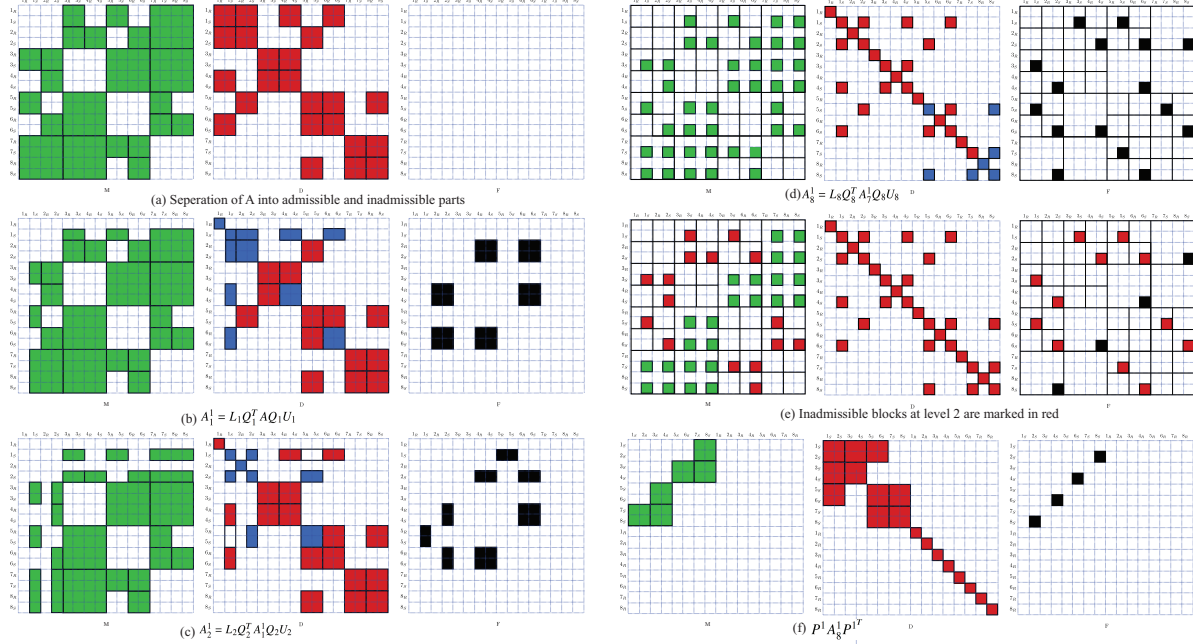


Fig. 9: [43, figs. 1 and 4-12] Illustration of the factorization algorithm of [43] on the $\mathcal{H}^2$ example of Fig. 7(b). (a) The separation of $A = M + D + F$ where M and D contain respectively all admissible and inadmissible blocks at level 1 and F contains the fill-ins to be generated. Note that A has been reordered such that for each cluster t , $I_t = [t_R, t_S]$ consisting of the redundant indices and skeletonization indices, respectively. (b) Skeletonization using Q_1 and elimination using L_1, U_1 for cluster 1. After the skeletonization, the elimination of block row/column 1_R introduces modifications to existing inadmissible blocks (shown in blue) and new fill-ins which are admissible (shown in black). (c) Skeletonization using Q_2 and elimination using L_2, U_2 for cluster 2. (d) Skeletonization using Q_8 and elimination using L_8, U_8 for cluster 8. (e) The nonzero blocks in M and F corresponding to level-2 inadmissible blocks are marked in red and will be moved to D next. (f) Permutation P^1 is applied to permute the diagonal blocks of redundant indices to the lower right corner, which is then LU factorized as $L_r^1 U_r^1$.

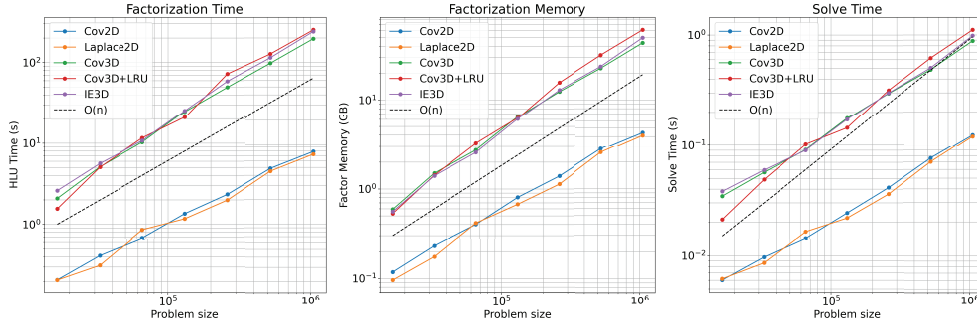


Fig. 10: [43, figs. 13 and 16] The factorization time, memory and solve time of the shared-memory parallel $\mathcal{H}^2$ algorithm of [43].

4.4 Parallelization of rank-structured matrix algorithm

Beyond the superior asymptotic complexities of rank-structured matrix algorithms shown in Table 4, their efficient implementation on modern parallel computers represents a core enabler for real-world large-scale multi-physics, multi-scale, and high-fidelity simulations. That said, these algorithms can face significant parallel scalability challenges as (a) different parallel data layouts can introduce vastly different communication patterns, (b) many algorithms feature

sequences of small and irregular-sized numerical linear algebra operations making them hard to achieve peak flop performance of modern hardware, and (c) many factorization algorithms involve complex data dependencies and recursive computation that are sequential in nature. In what follows, we review existing parallelization strategies for rank-structured matrices to tackle these challenges.

4.4.1 Distributed-memory process layout

When the matrix size n is large enough such that the entire rank-structured matrix cannot fit onto a single shared-memory machine, distributed-memory parallelism becomes essential to fully unlock the potential of these algorithms. The optimal parallel data layout should be able to simultaneously achieve (a) good load balance for matrices with general sizes and non-uniform ranks on non-power-of-two number of processes, and (b) minimal communication during construction, factorization and solve. Here we summarize a few existing parallel data layouts using examples of Fig. 11.

For a hierarchical matrix, the most common parallel layout is perhaps the 1D block layout of Fig. 11(a), which has been used in HODLR and HODBF matrices [58, 193], $\mathcal{H}$ matrices [132, 95], and $\mathcal{H}^2$ matrices [35, 227] for the construction phase and matrix-vector multiplication. This layout exhibits good load balance, but may require extra data redistribution in matrix-vector multiplication when the input vectors also follow the 1D block layout. Moreover, even this layout leads to manageable communication in the factorization phase for weak-admissible formats such as HODLR and HODBF matrices [193] (see Fig. 12 for the strong scaling performance of HODLR, HODBF and HSS-BF algorithms in the ButterflyPACK package [146]), they post significant challenges for recursive computation-based $\mathcal{H}$ and $\mathcal{H}^2$ factorization algorithms. In [219], a so-called lattice- $\mathcal{H}$ matrix representing a hybrid algorithm between BLR and $\mathcal{H}$ uses a 2D block-cyclic layout as shown in Fig. 11(b), which improves parallel efficiency of the factorization phase at the cost of introducing more flop operations. See Fig. 12 for the performance difference between 1D block and 2D block-cyclic layouts for the lattice- $\mathcal{H}$ algorithm in ButterflyPACK [146]. In addition, the HSS algorithm of [187] uses a tree-based parallel layout that simplifies the communication pattern during the HSS-ULV factorization and some strong scaling results are shown in Fig. 12 for HSS construction. Other parallel layouts have been used in parallel HIF [134], $\mathcal{H}^2$ [136] and BLR [60] factorization algorithms, etc.

When it comes down to the parallel layout of each individual off-diagonal block (e.g., the top-level off-diagonal block of Fig. 11(a)), one can use the 2D block-cyclic [187] or 1D block [132] layout for each low-rank factor, and a parallel FFT-like layout for each butterfly representation [183, 155]. See Fig. 11(d)-(e) for an illustration.

4.4.2 Fine-grained parallelism on CPU and GPU

Efficient parallel implementation of the algorithm (or its local workload of each MPI process) on shared-memory CPU and GPU architectures must take into consideration the irregular data structures in each phase of rank-structured matrix algorithm. For example, the matrix-vector multiplication of a $\mathcal{H}$ matrix involves an $\mathcal{O}(n/2^l)$ number of fine-grained GEMV operations of non-uniform sizes at each tree level l . As another example, the factorization of a HODLR matrix involves an $\mathcal{O}(n/2^l)$ number of fine-grained LU decompositions of variable-size diagonal blocks at each level l . Common implementation strategies are to flatten and marshal the tree metadata so that pertinent operations can be batched on GPUs or CPUs, and possible CPU-GPU data transfer can be minimized. This is particularly true for shared-basis algorithms such as HSS and $\mathcal{H}^2$ where even at higher level of the tree operations only involve small-sized blocks. These strategies have been leveraged in GPU implementation of BLR factorization [59, 115], HODLR factorization [56], $\mathcal{H}^2$ multiplication [45, 115, 227, 221], sketching-based $\mathcal{H}^2$ construction [44], and shared-memory CPU-based $\mathcal{H}^2$ factorization [43]. Needless to say, the same strategies implemented on GPUs can behave much better than those implemented on CPUs, see the example of $\mathcal{H}^2$ construction [44] in Fig. 8.

4.4.3 Strategies to tackle data dependency

The other parallelization challenge is the data dependency, particularly in the factorization phase, of many rank-structured matrix algorithms, e.g., the recursive computation-based $\mathcal{H}$ and $\mathcal{H}^2$ factorization algorithms [39, 92], the skeletonization-based $\mathcal{H}^2$ factorization algorithms [170], and the sketching-based $\mathcal{H}$ matrix construction algorithms

[139]. Common practices to exploit parallelism within data dependency are three folds: (a) One can leverage graph-based algorithms such as graph coloring to explore operations that can be simultaneously executed. This has been demonstrated for $\mathcal{H}^2$ factorization [136] and construction [127]. (b) Runtime scheduling systems such as StarPU [26], PARSEC [50] and OmpSs can be leveraged based on the DAG information of e.g., BLR [5, 50] and $\mathcal{H}$ matrix [219, 121, 7] algorithms. (c) One can potentially modify the original factorization algorithm to remove, to some extent, the data dependency. Examples include the lattice- $\mathcal{H}$ [219] algorithm that behaves as BLR (which is easier to parallelize) at the higher tree levels, and the $\mathcal{H}^2$ /BLR/HSS algorithms of [159, 197, 160] which precompute the augmented basis matrices due to fill-ins associated with inadmissible blocks.

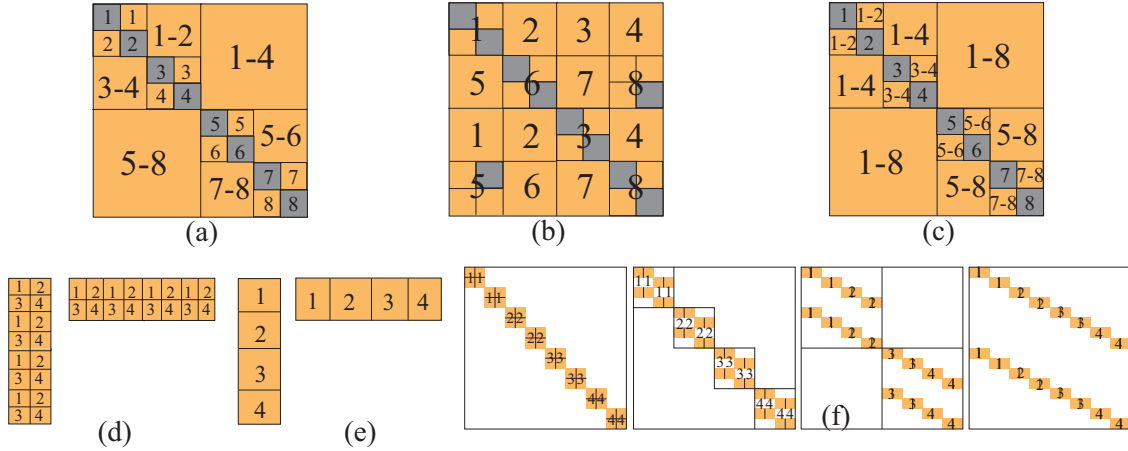


Fig. 11: (Top) Parallel process layouts for distributing a hierarchical matrix on 8 MPI processes: (a) 1D block layout for HODLR, HODBF matrices [58, 193], $\mathcal{H}$ matrices [132], and $\mathcal{H}^2$ matrices [35, 227], (b) 2D block-cyclic layout for lattice- $\mathcal{H}$ matrix algorithm [219] and BLR algorithm of [60], and (c) tree-based layout for HSS [187]. (Bottom) Parallel process layouts for distributing a single low-rank or butterfly compression on 4 MPI processes: (d) 2D block-cyclic layout for a low-rank representation used in [187], (e) 1D block layout for a low-rank representation used in [132], and (f) A parallel FFT-like layout for a butterfly representation used in [155, 193, 148].

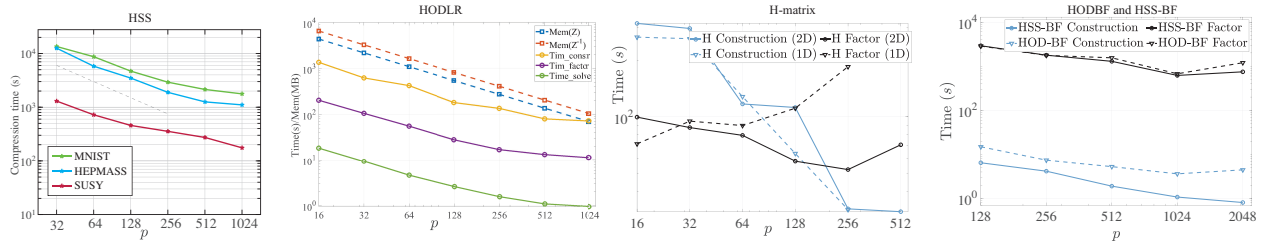


Fig. 12: Strong scaling (on distributed-memory systems) of various phases of several hierarchical matrix algorithms: HSS ([55, fig. 7]), HODLR, H matrix, HODBF and HSSBF.

5 Combining Structure-sparse and Data-sparse Formats

Many sparse matrices arising from PDE discretization on structured and unstructured space and time grids (see a few examples in Section 4.2.3) exhibit rank structures in their inverse or equivalent factorization [32, 53, 75, 73, 71], hence the rank-structured matrix algorithms introduced in Section 4 can be combined with sparse supernodal or multifrontal solvers to construct fast direct solvers or efficient preconditioners. Moreover, other data-sparse compression algorithms

such as lossy floating-point compression can also be combined with rank-structured matrix algorithms and sparse direct solvers. In what follows, we will focus on reviewing multifrontal sparse solvers with data-sparse compression. That said, there are also attempts to augment supernodal sparse solvers with algebraic compression, such as integration of HODLR [76] and BLR [175] in the PaStiX package [104], and the use of other low-rank compression techniques in supernodal-based Cholesky solver [51].

Algorithm 1 Sparse rank-structured multifrontal factorization using ZFP, BLR, HSS and HODBF matrix compression, followed by a GMRES iterative solve using the multifrontal factorization as an efficient preconditioner.

Input: $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$

Output: $x \approx A^{-1}b$

```

1:  $A \leftarrow P(D_r A D_c Q_c) P^\top$                                  $\triangleright$  scaling, and permutation for stability and fill reduction
2:  $A \leftarrow \hat{P} A \hat{P}^\top$                                            $\triangleright$  rank-reducing separator reordering
3: Build assembly tree: define  $I_\tau^s$  and  $I_\tau^u$  for every frontal matrix  $F_\tau$ 
4: for nodes  $\tau$  in assembly tree in topological order do
5:   if no compression then
6:     construct  $F_\tau$  and perform arithmetic as a dense matrix
7:   else if ZFP compression then
8:     construct  $F_\tau$  and perform arithmetic as a dense matrix
9:     compress as zfp matrix
10:  else if BLR compression then
11:     $F_\tau \leftarrow \begin{bmatrix} A(I_\tau^s, I_\tau^s) & A(I_\tau^s, I_\tau^u) \\ A(I_\tau^u, I_\tau^s) & 0 \end{bmatrix} \xleftrightarrow{\quad} T_{\tau_1} \xleftrightarrow{\quad} T_{\tau_2}$ 
12:     $P_\tau L_\tau U_\tau \leftarrow F_{11}$                                            $\triangleright$  LU with partial pivoting
13:     $F_{11}, F_{12}, F_{21} \leftarrow \text{BLR compress}(F_{11}, F_{12}, F_{21})$ 
14:     $F_{12} \leftarrow L_\tau^{-1} P_\tau^\top F_{12}$ 
15:     $F_{21} \leftarrow F_{21} U_\tau^{-1}$ 
16:     $T_\tau \leftarrow F_{22} - F_{21} F_{12}$                                            $\triangleright$  Schur update
17:  else if HODBF compression then
18:     $F_{11} \leftarrow \text{HODBF compress}(A(I_\tau^s, I_\tau^s) \xleftrightarrow{\quad} T_{\tau_1} \xleftrightarrow{\quad} T_{\tau_2})$ 
19:     $F_{11}^{-1} \leftarrow \text{HODBF invert}(F_{11})$ 
20:     $F_{12} \leftarrow \text{butterfly compress}(A(I_\tau^s, I_\tau^u) \xleftrightarrow{\quad} T_{\tau_1} \xleftrightarrow{\quad} T_{\tau_2})$ 
21:     $F_{21} \leftarrow \text{butterfly compress}(A(I_\tau^u, I_\tau^s) \xleftrightarrow{\quad} T_{\tau_1} \xleftrightarrow{\quad} T_{\tau_2})$ 
22:     $S \leftarrow \text{butterfly compress}(F_{21} F_{11}^{-1} F_{12})$                      $\triangleright$  Sketching-based construction
23:     $T_\tau \leftarrow \text{HODBF compress}(T_{\tau_1} \xleftrightarrow{\quad} T_{\tau_2} - S)$ 
24:  else if HSS compression then
25:     $F_\tau \leftarrow \text{HSS compress}\left(\begin{bmatrix} A(I_\tau^s, I_\tau^s) & A(I_\tau^s, I_\tau^u) \\ A(I_\tau^u, I_\tau^s) & 0 \end{bmatrix} \xleftrightarrow{\quad} T_{\tau_1} \xleftrightarrow{\quad} T_{\tau_2}\right)$      $\triangleright$  Sketching-based construction
26:     $U_\tau L_\tau V_\tau \leftarrow F_{11}$                                            $\triangleright$  HSS ULV
27:     $F_{12} \leftarrow L_\tau^{-1} U_\tau^\top F_{12}$ 
28:     $F_{21} \leftarrow F_{21} V_\tau^\top$ 
29:     $T_\tau \leftarrow F_{22} - F_{21} F_{12}$                                            $\triangleright$  Low-rank update
30:  end if
31: end for
32:  $x \leftarrow \text{GMRES}(A, b, M : u \leftarrow D_c Q_c P^\top \hat{P}^\top \text{ bwd-solve}(\text{fwd-solve}(\hat{P} P D_r v)))$ 

```

5.1 Overview of data-sparse multifrontal sparse solver and preconditioner

Data-sparse multifrontal solvers have a similar algorithmic workflow to exact multifrontal solvers, except that the dense frontal matrices and their arithmetic are compressed using various compression techniques depending on their sizes and underlying PDE characteristics.

The overall workflow is described in Algorithm 1. Just like exact multifrontal and supernodal solvers, the matrix A is first preprocessed as eq. (1) with diagonal scaling matrices D_r and D_c , and permutation matrices P_r and P_c . For example, a typical choice for the fill-in reduction permutation P_c is nested dissection, under which the frontal matrices can be viewed as graphs between the separators consisting of subsets of vertices of the original graph A . As explained in Section 4.3.1, the separators need to be further clustered to reveal rank structures inside each frontal matrix, via

graph partitioning algorithms such as recursive bisection [148, 86] or geometrical information if available [231]. This permutation is denoted $\hat{P}$ in Algorithm 1. Based on P_c , a structure called the assembly tree is constructed consisting of dense frontal matrices as the nodes (see Fig. 14 for an assembly tree example). For each non-root node, the frontal matrix has a 2×2 block structure $F_\tau = \begin{bmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{bmatrix}$, where F_{11} corresponds to fully summed variables I_τ^s , and F_{22} with indices I_τ^u is the trailing submatrix to be Schur complement updated by F_{11} and will contribute to the parent frontal matrix via extend-add. The data-sparse multifrontal solvers will then traverse the assembly tree in the topological order, and for each (non-leaf) frontal matrix perform factorization of F_{11} , update panels F_{12}/F_{21} , Schur complement update F_{22} as T , and propagate T into its parent using extend-add. For the root frontal matrix, only $F_{11} = F_\tau$ is factorized. Each frontal matrix can be compressed using any existing data-sparse compression algorithms such as lossy floating-point compression including ZFP [140] and SZ [65], and rank-structured matrix algorithms including $\mathcal{H}$, $\mathcal{H}^2$, HSS, BLR, HODLR, HODBF and their other variants. Most existing data-sparse multifrontal solvers use a single compression algorithm, but there also exist solvers that leverage more than one compression algorithms (i.e., hybrid or composite compression algorithms). Since lossy floating-point compression only work well from very small-sized frontal matrices or the inadmissible blocks inside each rank-structured matrix algorithm, we refer to data-sparse multifrontal sparse solver simply as rank-structured multifrontal sparse solvers in the rest of this chapter. Note that as the frontal matrix arithmetic are approximated by the compression methods using a prescribed tolerance, the multifrontal solver only computes a $M \approx A^{-1}$, which can be used either as a direct solver or a preconditioner (in e.g., GMRES) depending on the tolerance and application needs. In what follows, we review existing rank-structured multifrontal sparse solvers in literature.

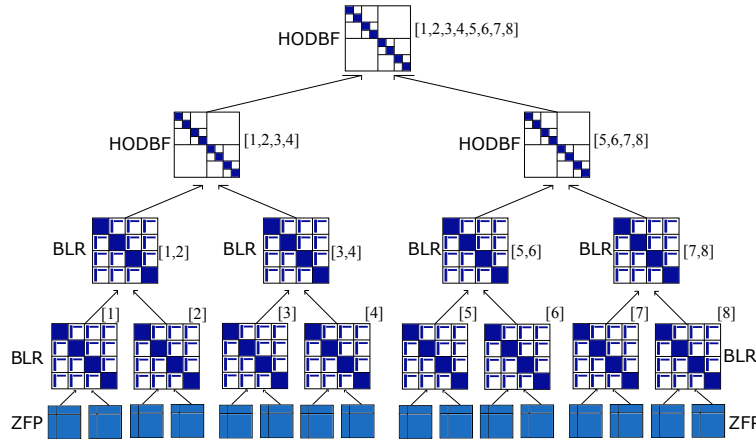


Fig. 13: [60, fig. 5] The assembly tree of rank-structured multifrontal solvers using different compression algorithms. The solver is parallelized using 8 MPI processes.

$\mathcal{H}$ and $\mathcal{H}^2$ matrices represent the first hierarchical matrix algorithms applied to frontal matrices. Earlier works consider the recursive computation-based factorization algorithms [93, 230, 30] which exhibit large computational overheads and difficulty in efficient parallelization [93], albeit with promising complexity claims. Modern $\mathcal{H}^2$ algorithms and variants such as IFMM and skeletonization-based algorithms have been recently introduced to multifrontal solvers [222, 223, 184, 231], representing an active research area. In addition to these strong-admissible algorithms, weak-admissible algorithms such as HSS [85, 86, 215, 211, 206, 204, 216] represent one of the foundational algorithms in rank-structured multifrontal solver package STRUMPACK [198]. As shown in Algorithm 1, HSS is used to compress F_τ , typically using sketching-based construction algorithms to ensure compression accuracy in fully algebraic settings. Next, F_{11} (which is also a HSS matrix) is factorized using HSS-ULV, and F_{22} can be low-rank updated into T_τ in an efficient manner. As HSS turns to become less efficient for high-dimensional problems (e.g. 3D PDEs), its high-dimensional variants such as HSS2D [212] and HIF [107, 77, 48] have been integrated into multifrontal solvers as well. Other weak-admissible formats such as HODLR and HODBF have also been integrated into multifrontal solvers [148] where the HODBF algorithm is well-suited to compress large frontal matrices arising from high-frequency wave equations. As shown in Algorithm 1, HODBF is used to compress F_{11} , F_{12} , F_{21} and T_τ separately, and the Schur complement update S is compressed using sketching-based algorithms [155]. Finally, BLR has also been a popular

problem	dim	rank $r(m)$				factor flops				solve flops/factor memory			
		HOD-BF	HSS	BLR	$\mathcal{H}^2$	HOD-BF	HSS	BLR	$\mathcal{H}^2$	HOD-BF	HSS	BLR	$\mathcal{H}^2$
Helmholtz	2	$\log m$	m	m	m	n	$n^{3/2}$	$n^{3/2}$	$n^{3/2}$	n	$n \log n$	$n \log n$	$n \log n$
	3	$m^{1/4}$	m	m	m	$n \log^2 n$	n^2	n^2	n^2	n	$n^{4/3}$	$n^{4/3}$	$n^{4/3}$
Poisson	2	$\log m$	$\log m$	1	1	n	n	$n \log n$	n	n	n	n	n
	3	$m^{1/4}$	$m^{1/2}$	1	1	$n \log^2 n$	$n^{4/3}$	$n^{4/3}$	n	n	n	$n \log n$	n

Table 5: Asymptotic complexity of rank-structured matrix (HODBF, HSS, BLR and $\mathcal{H}^2$) enhanced multifrontal solvers for 2D and 3D, Helmholtz and Poisson equations. Other variants like HIF [107], HSS2D [212], and multilevel BLR [16] are not shown here. The “big O” notation $O(\cdot)$ has been dropped. Here m denotes the size of a front and n is the size of the global sparse system.

method for rank-structured multifrontal solvers [167, 13, 20, 175, 15] which is available in both MUMPS [18] and STRUMPACK [198]. BLR has several variants [14, 60] each with different characteristics in flop performance, memory usage and communication complexity. We show the right-looking (RL)-BLR in Algorithm 1, which requires forming the frontal matrix in the dense format and performs the arithmetic in compressed representations. Other more memory efficient BLR variants such as hybrid BLR [60] have also been integrated into STRUMPACK. It’s worth mentioning that in addition to standard finite difference or finite element discretizations, rank-structured multifrontal solvers have also been developed for higher order discretization such as the hierarchical merging of Poincaré–Steklov (HPS) solvers [163, 101, 89] and spectral methods [195].

Remark on numerical stability. When one frontal matrix is dense without low-rank compression, F_{11} can be factorized as LU with partial pivoting P_τ which is numerically stable for ill-conditioned frontal matrices. However, rank-structured matrix algorithms can only allow partial pivoting on each leaf-level diagonal blocks, yielding these algorithms more prone to numerical instability. One exception is the BLR algorithm [167] which allows permutation of the blocks during factorization. In practice, a frontal matrix may need to be scaled or preconditioned before performing the hierarchical matrix arithmetic as demonstrated in [216, 77, 48].

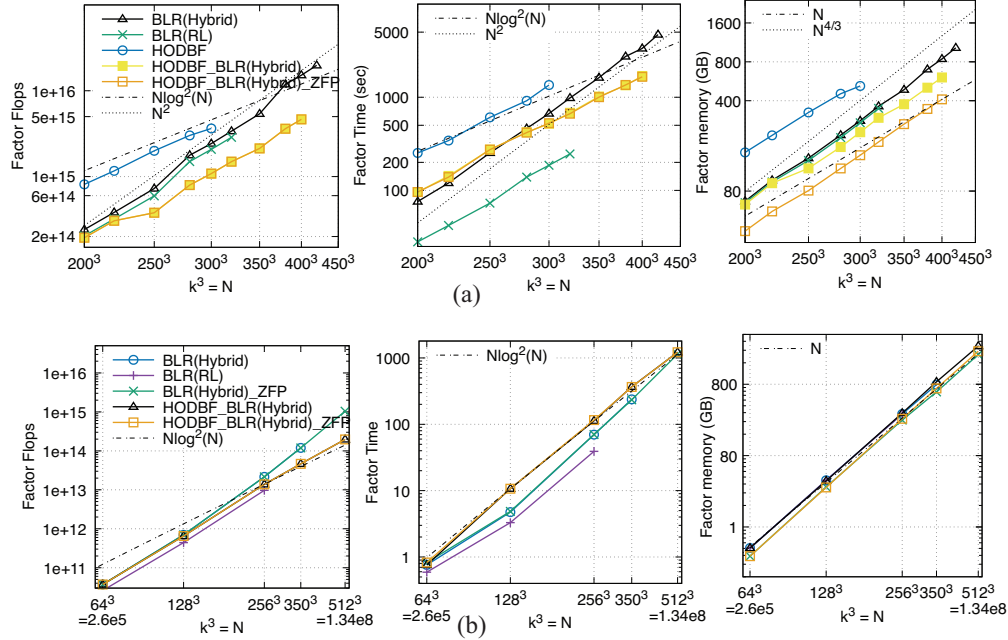


Fig. 14: [60, figs. 5 and 8] Characteristics of the factorization phase including flops, CPU time and memory for a multifrontal sparse solver with composite compression algorithms using 64 NERSC Cori CPU nodes.

Remark on computational complexity. The asymptotic complexity of rank-structured multifrontal solvers is closely related to the numerical rank of each frontal matrix compressed by a specific rank-structured matrix algorithm. As the

frontal matrix represents numerical Green’s function of the governing PDEs, acting on separators that are orthogonal or parallel to each other, the rank behavior can be estimated to derive the overall sparse solver complexity. In Table 5 we summarize the asymptotic complexities for the factor flops, solve flops and factor memory of different rank-structured multifrontal solvers using two classes of 2D and 3D PDEs, namely Poisson and high-frequency Helmholtz. We list the rank of each frontal matrix of size $m \times m$, $r(m)$, for HODBF, HSS, BLR and $\mathcal{H}^2$ algorithms as well as the overall solver complexity in terms of the sparse matrix size n . It’s worth noting that for 3D high-frequency Helmholtz, the theoretical rank estimate is $r(m) = \mathcal{O}(m)$ for HSS, BLR, and $\mathcal{H}^2$, etc [148, 71], which leads to no theoretical asymptotic complexity reduction compared to $\mathcal{O}(n^2)$. However, one often empirically observes $r(m) = \mathcal{O}(m^{0.5})$ when $n \leq 400^3$ [204, 206, 205], leading to an empirically reduced solver complexity. While the optimal asymptotic complexity depends on PDE types, dimensionality, problem sizes and compression algorithms, we remark that in practice there doesn’t necessarily exist a single compression algorithm that yields the best compression performance for all frontal matrices in a multifrontal solver. As such, hybrid or composite compression-based [60] multifrontal solvers may become necessary. As an example, we illustrate a composite multifrontal solver [60] in Fig. 13 where small-sized frontal matrices are compressed as ZFP, small-to-medium-sized ones are compressed using BLR (hybrid BLR or RL BLR), and large ones are compressed using HODBF. Fig. 14 demonstrates that the fully composite solver (labeled as “HODBF.BLR(Hybrid).ZFP”) is capable of handling $n = 420^3 = 74,088,000$ for 3D high-frequency wave equations and $n = 512^3 = 134,217,728$ for 3D singularly perturbed reaction-diffusion equations, using 2048 NERSC Cori CPU cores.

Remark on distributed-memory and fine-grained parallelization. Distributed-memory parallelization of rank-structured multifrontal solvers represents a core technique to fully unlock the potential of these solvers for large-scale PDE simulations and machine learning computation. Most distributed-memory parallel multifrontal solvers follow a nested parallelization design pattern, which is briefly explained as follows: At the outer layer, the assembly tree is distributed using a tree parallelism approach [19]. The root node frontal matrix is computed on all processes in the root MPI communicator; the root MPI communicator is split into two smaller ones, each handling one of the children frontal matrices at the next level; this procedure continues until a frontal matrix (and the corresponding subtree) is fully occupied by one MPI process. Fig. 13 shows the distribution of the assembly tree with 8 MPI processes. Note that each parent MPI communicator doesn’t need to be split into two approximately equal-sized communicators. One can estimate the workload of its child (and the associated subtree) and use the so-called proportional mapping to assign communicator sizes proportional to the estimated workloads [19, 206]. At the inner layer of the nested parallelization, each rank-structured frontal matrix occupied by more than one MPI process can be parallelized using the data layouts explained in Section 4.4.1, and each dense frontal matrix can be parallelized using e.g., a 2D block-cyclic layout. The above-described nested parallelization strategy has been utilized in several rank-structured multifrontal solvers, including BLR-based [59, 167], HSS-based [85, 206, 204], HODBF-based [148] and composite compression-based [60] solvers. Fig. 15(a) shows the strong scaling performance of different phases of a HSS-based multifrontal solver in STRUMPACK [85] for a sparse matrix tdr455k from accelerator simulations using up to 1152 CPU cores.

Fine-grained parallelism inside each frontal matrix can naturally leverage the techniques summarized in Section 4.4.2. Beyond that, this kind of parallelism can also be exploited across dense [87] or rank-structured frontal matrices, particularly at lower levels of the assembly tree consisting of large numbers of small-sized frontal matrices. Examples include shared-memory HSS-multifrontal solvers [86] and GPU BLR-multifrontal solvers [59]. In addition, GPUs have been exploited for several two-level multifrontal solvers such as [222, 123]. For large-sized frontal matrices, it’s worth mentioning that distributed-memory multi-GPU parallelization of a single frontal matrix has also been recently developed leveraging batched vendor GPU BLAS libraries and GPU-aware MPI libraries [59]. Fig. 15(b) demonstrates the performance of the BLR-multifrontal solver [59] compared with exact supernodal and multifrontal solvers on 2048 CPU cores or 128 GPUs on NERSC Perlmutter.

6 Available Software for Distributed Memory Parallel Computers

In this section, we list in table 6 several direct solver packages (open-source for most of them) leveraging structure-sparse and/or data-sparse algorithms. Here we only mention several packages designed with distributed-memory parallelism, and implemented with HPC programming languages such as C, C++, Fortran or Julia. Note that several of them also provide support for Python or Matlab interfaces. A more complete list can be found at <https://portal.nersc.gov/>

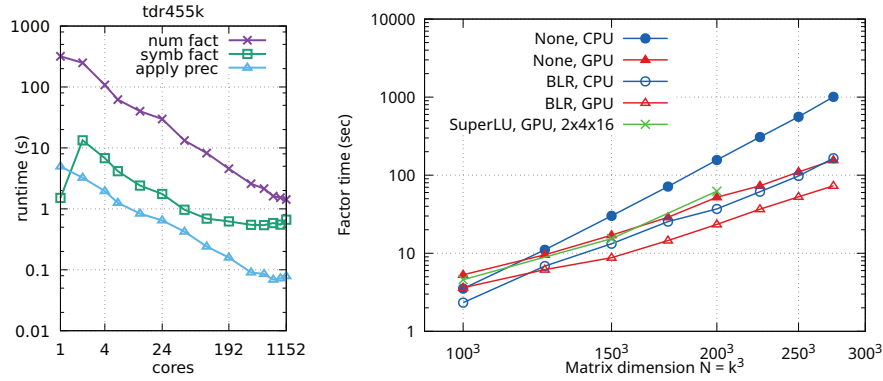


Fig. 15: (a) [60, fig. 6] Strong scaling of the parallel HSS-multifrontal solver of STRUMPACK. (b) [59, fig. 1] Performance comparison of the exact multifrontal solver and BLR-multifrontal solver in STRUMPACK and the supernodal solver SuperLU_DIST on multiple CPU and GPU nodes.

project/sparse/superlu/SparseDirectSurvey.pdf for structure-sparse solvers and https://github.com/gchavez2/awesome_hierarchical_matrices for data-sparse solvers.

Software	GPU	Matrix Type	Data-sparse	Programming Language	Typical Application
ASKIT [25, 58]	No	Dense	Yes	C++	Kernel
AHMED [4]	No	Dense/Sparse	Yes	C++	IE/FEM
ButterflyPACK [146]	No	Dense	Yes	Fortran/C++/Python	IE/Kernel/Optimization
DINFMM [66, 136]	No	Dense	Yes	Julia	IE
h2opus [99, 227]	Yes	Dense	Yes	C++	IE/Kernel/Optimization
HiCMA [105]	Yes	Dense	Yes	C/C++	IE/Kernel
HLibPro [106]	No	Dense/Sparse	Yes	C++	IE/Kernel/PDE
MUMPS [172, 18]	Yes	Sparse	Yes	C/Fortran	Any sparse
mxc [173, 160]	Yes	Dense	Yes	C++	IE/Kernel
PaStiX [181, 104]	Yes	Sparse	Yes	C/Fortran/Python/Julia	Any sparse
PARDISO [179]	No	Sparse	No	C/C++/Fortran/Python/Matlab	Any sparse
STRUMPACK [198]	Yes	Dense/Sparse	Yes	C++/Fortran/Python	IE/Kernel/Optimization/Any sparse
SuperLU_DIST [199, 135]	Yes	Sparse	No	C/C++/Fortran/Python/Julia/Matlab	Any sparse

Table 6: List (in alphabetical order) of some distributed-memory direct solver packages leveraging structure sparsity and/or data sparsity.

Among the packages in table 6, SuperLU_DIST and PARDISO are exact direct solvers that leverage only data-sparse algorithms and can apply to any sparse matrices. Other such packages not listed here include PanguLU [178], symPACK [201] and WSMP [209], etc. On the other hand, the other packages in table 6 leveraging data-sparse algorithms for dense matrices implement a variety of rank-structured matrix algorithms described in section 4.1 and usually work as efficient preconditioners. Some of these packages are designed for a particular application such as kernel methods in machine learning and statistics, integral equation (IE) or finite-element method (FEM) matrices, and thus provide additional support at the application layer (e.g., PDE discretization, quadrature, hyperparameter optimization, etc.). Note that there are several dense direct solver packages that only support construction, rather than factorization, of a rank-structured matrix, which we don't list here. Finally, MUMPS, PaStiX, STRUMPACK, AHMED and HLibPro in the table combine structure-sparse and data-sparse algorithms for sparse matrices exhibiting low-rank structures, often arising from PDE or optimization problems. Note that MUMPS, PaStiX and STRUMPACK can behave as exact direct solvers when their data-sparse algorithms are turned off.

7 Open Problems

Despite tremendous progress in parallel algorithms for both structure-sparse and data-sparse direct solvers, there remain many open problems, both theoretical and practical. We list some of them in the following.

- *GPU-resident solvers.* Today's GPU memory is orders of magnitude larger than that of ten years ago. As such, many application codes are moved entirely to GPUs, avoiding the data transfer cost between CPU and GPU. This requires the linear solvers to be running on GPUs as well. The open problems are mostly in the preprocessing steps that are largely on CPUs. These are all combinatorial algorithms and much less progress is achieved on GPUs.
- *Distributed memory symmetric indefinite sparse direct solvers.* This capability is sorely needed by the optimization community. The symmetric LDL^T factorization is highly desirable in numerical optimization methods that use second-order information, as it exposes matrix inertia needed to assess local convexity and KKT optimality. As of now, a robust and widely used algorithm is MA57 [69], but there is no reliable distributed memory code.
- *Communication analysis for data-sparse parallel algorithms.* While communication analysis is well-established for exact dense and structure-sparse direct solvers, there is a dearth of research concerning data-sparse direct solvers. In fact, there exists only a limited body of latency–bandwidth model-based analysis for the construction phase [132, 155], with none available for the factorization phase. Accordingly, communication avoiding algorithms remain unexplored for rank-structured matrix algorithms. Moreover, distributed-memory parallel algorithms and dynamical runtime scheduling techniques haven't been reported for sketching-based construction and factorization algorithms for strong-admissible rank-structured matrix algorithms.
- *Data-sparse in tensor computations.* The efficiency of rank-structured matrix algorithms for high-dimensional applications—such as 3D integral equations, kernel matrices, and quantum chemistry—remains a significant challenge with substantial potential for optimization. Tensor-based extensions of these algorithms represent a promising frontier for reducing both the constant prefactors and the asymptotic complexities of existing fast direct solvers. Recent efforts include the development of tensor-structured $\mathcal{H}$ -matrices [131] and butterfly algorithms [152, 118], tensor-train-based solvers for integral equations [62, 174], as well as tensor-based approaches for wave equations with varying frequencies or coefficients. However, these methods primarily target structured data, such as PDEs discretized on regular grids; their applicability to unstructured data remains an open problem. Furthermore, while several parallel algorithms for individual tensor compressions exist [28], fully parallelized tensor-based direct solvers have yet to be developed.
- *Data-sparse rank analysis for matrix inverse.* While the rank behavior of frontal matrices in sparse solvers for PDEs is relatively well understood, the rank behavior of inverses for integral equation matrices, particularly using strong-admissible-based rank structured algorithms, has been addressed primarily through empirical studies. Theoretical results have only been established for Laplacian kernels with quasi-uniform discretization [74, 72]. Comparing with the rank structure of the integral equation matrices, the ranks in the inverse often exhibit mild growth. This phenomenon is attributed to both the accumulation of numerical errors and the increasingly complex physics (e.g., scattering matrices in wave problems) encoded within the Schur complements. Consequently, rigorous analytical tools for characterizing rank growth in the inverses of integral equation matrices remain largely unavailable.
- *Error and Spectral analysis for data-sparse matrices.* Error analysis for rank-structured matrices represents an indispensable tool for understanding the reliability of these algorithms and providing practical guidance on algorithm parameter choices. Backward stability analysis exists for certain weak-admissible formats such as HSS [162, 210], but remains largely unavailable for most rank-structured matrix algorithms. Moreover, the effectiveness of rank-structured preconditioners for ill-conditioned systems, such as recent $\mathcal{H}^2$ preconditioners [160, 197], requires rigorous spectral analysis tools.

Acknowledgements This work was supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research's Applied Mathematics Competitive Portfolios program and Scientific Discovery through Advanced Computing (SciDAC) program through the FASTMath Institute, under contract number DE-AC02-05CH11231.

References

1. Abdelfattah, A., Beams, N., Carson, R., Ghysels, P., Kolev, T., Stitt, T., Vargav, A., Tomov, S., Dongarra, J.: MAGMA: Enabling exascale performance with accelerated BLAS and LAPACK for diverse GPU architectures. The International Journal of High

- Performance Computing Applications **38**(5), 468–490 (2024). doi:10.1177/10943420241261960
2. Abdelfattah, A., Ghysels, P., Boukaram, W., Tomov, S., Li, X.S., Dongarra, J.: Addressing irregular patterns of matrix computations on GPUs and their impact on applications powered by sparse direct solvers. In: SC22: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–14. IEEE (2022). doi:10.1109/SC41404.2022.00031
 3. Agullo, E., Demmel, J., Dongarra, J., Hadri, B., Kurzak, J., Langou, J., Ltaief, H., Luszczek, P., Tomov, S.: Numerical linear algebra on emerging architectures: The PLASMA and MAGMA projects. J. Phys.: Conf. Ser. **180**(1) (2009)
 4. AHMED. <https://www.wr.uni-bayreuth.de/en/software/ahmed/index.html>
 5. Al-Harthi, N., Alomairy, R., Akbudak, K., Chen, R., Ltaief, H., Bagci, H., Keyes, D.: Solving acoustic boundary integral equations using high performance tile low-rank LU factorization. In: International Conference on High Performance Computing, pp. 209–229. Springer (2020)
 6. Alger, N.V.: Data-scalable Hessian preconditioning for distributed parameter PDE-constrained inverse problems. The University of Texas at Austin (2019)
 7. Aliaga, J.I., Carratalá-Sáez, R., Kriemann, R., Quintana-Ortí, E.S.: Task-parallel LU factorization of hierarchical matrices using OmpSs. In: 2017 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), pp. 1148–1157. IEEE (2017)
 8. Ambartsumyan, I., Boukaram, W., Bui-Thanh, T., Ghattas, O., Keyes, D., Stadler, G., Turkiyyah, G., Zampini, S.: Hierarchical matrix approximations of Hessians arising in inverse problems governed by PDEs. SIAM Journal on Scientific Computing **42**(5), A3397–A3426 (2020)
 9. Ambikasaran, S., Darve, E.: An $O(N \log N)$ fast direct solver for partial hierarchically semi-separable matrices: with application to radial basis function interpolation. Journal of Scientific Computing **57**(3), 477–501 (2013)
 10. Ambikasaran, S., Darve, E.: The inverse fast multipole method. arXiv preprint arXiv:1407.1572 (2014)
 11. Ambikasaran, S., Foreman-Mackey, D., Greengard, L., Hogg, D.W., O’Neil, M.: Fast direct methods for Gaussian processes. IEEE transactions on pattern analysis and machine intelligence **38**(2), 252–265 (2015)
 12. Ambikasaran, S., O’Neil, M., Singh, K.R.: Fast symmetric factorization of hierarchical matrices with applications. arXiv preprint arXiv:1405.0223 (2014)
 13. Amestoy, P., Ashcraft, C., Boiteau, O., Buttari, A., L’Excellent, J.Y., Weisbecker, C.: Improving multifrontal methods by means of block low-rank representations. SIAM J. Sci. Comput. **37**(3), A1451–A1474 (2015)
 14. Amestoy, P., Buttari, A., L’Excellent, J.Y., Mary, T.: On the complexity of the block low-rank multifrontal factorization. SIAM Journal on Scientific Computing **39**(4), A1710–A1740 (2017)
 15. Amestoy, P.R., Buttari, A., L’Excellent, J.Y., Mary, T.: Performance and scalability of the block low-rank multifrontal factorization on multicore architectures. ACM Trans. Math. Softw. **45**(1) (2019). doi:10.1145/3242094
 16. Amestoy, P.R., Buttari, A., L’Excellent, J.Y., Mary, T.A.: Bridging the gap between flat and hierarchical low-rank matrix formats: the multilevel block low-rank format. SIAM J. Sci. Comput. **41**(3), A1414–A1442 (2019). doi:10.1137/18M1182760
 17. Amestoy, P.R., Duff, I.S., L’Excellent, J.Y., Li, X.S.: Analysis and comparison of two general sparse solvers for distributed memory computers. ACM Trans. Math. Softw. **27**(4), 388–421 (2001). doi:10.1145/504210.504212. URL <https://doi.org/10.1145/504210.504212>
 18. Amestoy, P.R., Duff, I.S., L’Excellent, J.Y., Koster, J.: MUMPS: a general purpose distributed memory sparse solver. In: International Workshop on Applied Parallel Computing, pp. 121–130. Springer (2000)
 19. Amestoy, P.R., Duff, I.S., L’Excellent, J.Y., Koster, J.: A fully asynchronous multi-frontal solver using distributed dynamic scheduling. SIAM Journal on Matrix Anal. Appl. **23**, 15–41 (2001). doi:10.1137/S0895479899358194
 20. Aminfar, A., Ambikasaran, S., Darve, E.: A fast block low-rank dense solver with applications to finite-element matrices. J. Comput. Phys. **304**, 170–188 (2016)
 21. Anderson, E., Saad, Y.: Solving sparse triangular linear systems on parallel computers. Int. J. High Speed Comput. **1**(1), 73–95 (1989). doi:10.1142/S0129053389000056. URL <https://doi.org/10.1142/S0129053389000056>
 22. Angleitner, N., Faustmann, M., Melenk, J.M.: $\mathcal{H}$ -inverses for RBF interpolation. Advances in Computational Mathematics **49**, 1–46 (2021). URL <https://api.semanticscholar.org/CorpusID:237540970>
 23. Anzt, H., Cojocan, T., Flegar, G., Göbel, F., Grützmacher, T., Nayak, P., Ribizel, T., Tsai, Y.M., Quintana-Ortí, E.S.: Ginkgo: a modern linear operator algebra framework for high performance computing. ACM Transactions on Mathematical Software **48**(1), 2:1–2:33 (2022). doi:10.1145/3480935. URL <https://doi.org/10.1145/3480935>
 24. Ashcraft, C., Buttari, A., Mary, T.: Block low-rank matrices with shared bases: potential and limitations of the BLR² format. SIAM Journal on Matrix Analysis and Applications **42**(2), 990–1010 (2021)
 25. ASKIT. <https://padas.oden.utexas.edu/libaskit>
 26. Augonnet, C., Thibault, S., Namyst, R., Wacrenier, P.A.: StarPU: a unified platform for task scheduling on heterogeneous multicore architectures. Concurrency and Computation: Practice and Experience **23**(2), 187–198 (2011). doi:10.1002/cpe.1631. URL <https://inria.hal.science/inria-00550877>
 27. Ballard, G., Carson, E., Demmel, J., Hoemmen, M., Knight, N., Schwartz, O.: Communication lower bounds and optimal algorithms for numerical linear algebra. Acta Numerica **23**, 1–155 (2014). doi:10.1017/S0962492914000038
 28. Ballard, G., Kolda, T.G.: Tensor decompositions for data science. Cambridge University Press (2025)
 29. Barnett, A., Wu, B., Veerapaneni, S.: Spectrally accurate quadratures for evaluation of layer potentials close to the boundary for the 2D Stokes and Laplace equations. SIAM Journal on Scientific Computing **37**(4), B519–B542 (2015)
 30. Bebendorf, M.: Approximation of boundary element matrices. Numerische Mathematik **86**(4), 565–589 (2000). doi:10.1007/PL00005410
 31. Bebendorf, M., Grzhibovskis, R.: Accelerating Galerkin BEM for linear elasticity using adaptive cross approximation. Mathematical Methods in the Applied Sciences **29**(14), 1721–1747 (2006). doi:10.1002/mma.759
 32. Bebendorf, M., Hackbusch, W.: Existence of $\mathcal{H}$ -matrix approximants to the inverse FE-matrix of elliptic operators with L_∞ -coefficients. Numerische Mathematik **95**(1), 1–28 (2003)

33. Bebendorf, M., Venn, R.: Constructing nested bases approximations from the entries of non-local operators. *Numerische Mathematik* **121**(4), 609–635 (2012)
34. Belli, R., Hoefer, T.: Notified access: Extending remote memory access programming models for producer-consumer synchronization. In: *IEEE International Parallel and Distributed Processing Symposium*, pp. 871–881. IEEE (2015)
35. Bendoraityte, J., Börm, S.: Distributed $\mathcal{H}^2$ -matrices for non-local operators. *Comput. Vis. Sci* **11**, 237–249 (2008)
36. Bhatia, N., Vandana: Survey of nearest neighbor techniques (2010). URL <https://arxiv.org/abs/1007.0085>
37. Birdsall, C.K., Langdon, A.B.: *Plasma physics via computer simulation*. CRC press (2018)
38. Blackford, L.S., Choi, J., D'Azevedo, E., Demmel, J., Dhillon, I., Dongarra, J., Hammarling, S., Henry, G., Petitet, A., Stanley, K., Walker, D., Whaley, R.C.: *ScaLAPACK users' guide*. SIAM, Philadelphia (1997). <http://www.netlib.org/scalapack>
39. Börm Steffen Grasedyck, L., Hackbusch, W.: Introduction to hierarchical matrices with application. *Eng. Anal. Bound. Elem.* **27**, 405–422 (2003)
40. Börm, S., Reimer, K.: Efficient arithmetic operations for rank-structured matrices based on hierarchical low-rank updates. *Computing and Visualization in Science* **16**, 247–258 (2013). doi:10.1007/s00791-014-0236-7
41. Bosilca, G., Bouteiller, A., Danalis, A., Herault, T., Lemarinier, P., Dongarra, J.: DAGuE: A generic distributed DAG engine for High Performance Computing. *Parallel Comput.* **38**(1–2), 37–51 (2012). doi:10.1016/j.parco.2011.10.003. URL <https://icl.utk.edu/parsec/>
42. Boukaram, W., Hong, Y., Liu, Y., Shi, T., Li, X.S.: Batched sparse direct solver design and evaluation in SuperLU_DIST. *The International Journal of High Performance Computing Applications* **38**(6), 585–598 (2024). doi:10.1177/10943420241268200
43. Boukaram, W., Keyes, D., Li, S., Liu, Y., Turkiyyah, G.: Linear complexity $\mathcal{H}^2$ direct solver for fine-grained parallel architectures. *arXiv preprint arXiv:2509.11152* (2025)
44. Boukaram, W., Liu, Y., Ghysels, P., Li, X.S.: Adaptive Sketching Based Construction of $\mathcal{H}^2$ Matrices on GPUs. In: *The 26th IEEE International Workshop on Parallel and Distributed Scientific and Engineering Computing (PDSEC 2025)*. IEEE (2025). Best Paper Award
45. Boukaram, W.H., Turkiyyah, G., Keyes, D.: Hierarchical matrix operations on GPUs: Matrix-vector multiplication and compression. *ACM Transactions on Mathematical Software* **45**(1), 1–28 (2019). doi:10.1145/3232850
46. Bradley, A.M.: A Hybrid Multithreaded Direct Sparse Triangular Solver, pp. 13–22. doi:10.1137/1.9781611974690.ch2
47. Börm, S.: Directional $\mathcal{H}^2$ -matrix compression for high-frequency problems. *Numer. Linear Algebra Appl.* **24**(6), e2112 (2017). doi:10.1002/nla.2112
48. Cambier, L., Chen, C., Boman, E.G., Rajamanickam, S., Tuminaro, R.S., Darve, E.: An algebraic sparsified nested dissection algorithm using low-rank approximations. *SIAM Journal on Matrix Analysis and Applications* **41**(2), 715–746 (2020)
49. Candès, E., Demanet, L., Ying, L.: A fast butterfly algorithm for the computation of Fourier integral operators. *SIAM Multiscale Model. Simul.* **7**(4), 1727–1750 (2009)
50. Cao, Q., Pei, Y., Herault, T., Akbudak, K., Mikhalev, A., Bosilca, G., Ltaief, H., Keyes, D., Dongarra, J.: Performance analysis of tile low-rank Cholesky factorization using PARSEC instrumentation tools. In: *2019 IEEE/ACM International Workshop on Programming and Performance Visualization Tools (ProTools)*, pp. 25–32. IEEE (2019)
51. Chadwick, J.N., Bindel, D.S.: An efficient solver for sparse linear systems based on rank-structured Cholesky factorization. *arXiv preprint arXiv:1507.05593* (2015)
52. Chandrasekaran, S., Dewilde, P., Gu, M., Lyons, W., Pals, T.: A fast solver for HSS representations via sparse matrices. *SIAM Journal on Matrix Analysis and Applications* **29**(1), 67–81 (2007)
53. Chandrasekaran, S., Dewilde, P., Gu, M., Somasunderam, N.: On the numerical rank of the off-diagonal blocks of Schur complements of discretized elliptic PDEs. *SIAM Journal on Matrix Anal. Appl.* **31**, 2261–2290 (2010). doi:10.1137/090775932
54. Chandrasekaran, S., Gu, M., Pals, T.: A fast ULV decomposition solver for hierarchically semiseparable representations. *SIAM Journal on Matrix Analysis and Applications* **28**(3), 603–622 (2006)
55. Chávez, G., Liu, Y., Ghysels, P., Li, X.S., Rebroya, E.: Scalable and memory-efficient kernel ridge regression. In: *2020 IEEE International parallel and distributed processing symposium (IPDPS)*, pp. 956–965. IEEE (2020)
56. Chen, C., Martinsson, P.G.: Solving linear systems on a GPU with hierarchically off-diagonal low-rank approximations. In: *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–15. IEEE (2022)
57. Chenhan, D.Y., March, W.B., Biros, G.: An $n \log n$ parallel fast direct solver for kernel matrices. In: *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 886–896. IEEE (2017)
58. Chenhan, D.Y., March, W.B., Xiao, B., Biros, G.: INV-ASKIT: a parallel fast direct solver for kernel matrices. In: *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 161–171. IEEE (2016)
59. Claus, L., Ghysels, P., Boukaram, W.H., Li, X.S.: A graphics processing unit accelerated sparse direct solver and preconditioner with block low rank compression
60. Claus, L., Ghysels, P., Liu, Y., Nhan, T.A., Thirumalaisamy, R., Bhalla, A.P.S., Li, S.: Sparse approximate multifrontal factorization with composite compression methods. *ACM Transactions on Mathematical Software* **49**(3), 1–28 (2023)
61. Corona, E., Martinsson, P.G., Zorin, D.: An $O(N)$ direct solver for integral equations on the plane. *Applied and Computational Harmonic Analysis* **38**(2), 284–317 (2015)
62. Corona, E., Rahmian, A., Zorin, D.: A tensor-train accelerated solver for integral equations in complex geometries. *J. Comput. Phys.* **334**, 145–169 (2017)
63. Coulier, P., Pouransari, H., Darve, E.: The inverse fast multipole method: using a fast approximate direct solver as a preconditioner for dense linear systems. *SIAM Journal on Scientific Computing* **39**(3), A761–A796 (2017)
64. Davis, T.A.: Algorithm 832: UMFPACK V4. 3—an unsymmetric-pattern multifrontal method. *ACM Trans. Math. Softw.* **30**(2), 196–199 (2004)
65. Di, S., Cappello, F.: Fast error-bounded lossy HPC data compression with SZ. In: *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 730–739. IEEE (2016)

66. DINFMM. <https://github.com/Tianyu-Liang/DINFMM/tree/main>
67. Ding, N., Liu, Y., Williams, S., Li, X.S.: A message-driven, multi-GPU parallel sparse triangular solver. In: Proceedings of the 2021 SIAM Conference on Applied and Computational Discrete Algorithms (ACDA21), pp. 147–159. doi:10.1137/1.9781611976830.14
68. Ding, N., Williams, S., Liu, Y., Li, X.S.: Leveraging one-sided communication for sparse triangular solvers. In: Proceedings of the 2020 SIAM Conference on Parallel Processing for Scientific Computing (PP), pp. 93–105. doi:10.1137/1.9781611976137.9
69. Duff, I.S.: Ma57—a code for the solution of sparse symmetric definite and indefinite systems. ACM Trans. Math. Softw. **30**(2), 118–144 (2004). doi:10.1145/992200.992202. URL <https://doi.org/10.1145/992200.992202>
70. Engquist, B., Ying, L.: Fast directional multilevel algorithms for oscillatory kernels. SIAM J. Sci. Comput. **29**(4), 1710–1737 (2007)
71. Engquist, B., Zhao, H.: Approximate separability of the Green’s function of the Helmholtz equation in the high frequency limit. Communications on Pure and Applied Mathematics **71**(11), 2220–2274 (2018)
72. Faustmann, M., Melenk, J., Praetorius, D.: Existence of $\mathcal{H}$ -matrix approximants to the inverses of BEM matrices: The simple-layer operator. Mathematics of Computation **85**(297), 119–152 (2016)
73. Faustmann, M., Melenk, J.M., Praetorius, D.: A new proof for existence of H-matrix approximants to the inverse of FEM matrices: the Dirichlet problem for the Laplacian. In: Spectral and High Order Methods for Partial Differential Equations-ICOSAHOM 2012: Selected papers from the ICOSAHOM conference, June 25–29, 2012, Gammarth, Tunisia, pp. 249–259. Springer (2013)
74. Faustmann, M., Melenk, J.M., Praetorius, D.: Existence of $\mathcal{H}$ -matrix approximants to the inverse of BEM matrices: the hyper-singular integral operator. Ima Journal of Numerical Analysis **37**, 1211–1244 (2015). URL <https://api.semanticscholar.org/CorpusID:116945940>
75. Faustmann, M., Melenk, J.M., Praetorius, D.: $\mathcal{H}$ -matrix approximability of the inverses of FEM matrices. Numer. Math. **131**(4), 615–642 (2015). doi:10.1007/s00211-015-0706-9. URL <https://doi.org/10.1007/s00211-015-0706-9>
76. Faverge, M., Pichon, G., Ramet, P., Roman, J.: On the use of H-matrix arithmetic in PaStiX: a preliminary study. In: Workshop on Fast Direct Solvers. Toulouse, France (2015). URL <https://inria.hal.science/hal-01187882>
77. Feliu-Fabà, J., Ho, K.L., Ying, L.: Recursively preconditioned hierarchical interpolative factorization for elliptic partial differential equations. Communications in Mathematical Sciences **18**(1), 91–108 (2020)
78. Feliu-Fabà, J., Ying, L.: Approximate inversion of discrete Fourier integral operators. J. Comput. Phys. **446**, 110654 (2021)
79. Feng, Y., Xiao, J., Gu, M.: Flip-flop spectrum-revealing QR factorization and its applications to singular value decomposition. Electron. Trans. Numer. Anal. **51**, 469–494 (2019). doi:10.1553/etna_vol51s469
80. Fu, X., Zhang, B., Wang, T., Li, W., Lu, Y., Yi, E., Zhao, J., Geng, X., Li, F., Zhang, J., Jin, Z., Liu, W.: PanguLU: a scalable regular two-dimensional block-cyclic sparse direct solver on distributed heterogeneous systems. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC ’23. Association for Computing Machinery, New York, NY, USA (2023). doi:10.1145/3581784.3607050
81. Gates, M., Kurzak, J., Charara, A., YarKhan, A., Dongarra, J.: SLATE: Design of a Modern Distributed and Accelerated Linear Algebra Library. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC ’19. Association for Computing Machinery, New York, NY, USA (2019). doi:10.1145/3295500.3356223
82. Geoga, C.J., Anitescu, M., Stein, M.L.: Scalable Gaussian process computations using hierarchical matrices. Journal of Computational and Graphical Statistics **29**(2), 227–237 (2020)
83. George, A.: Nested dissection of a regular finite element mesh. SIAM J. Numerical Analysis **10**, 345–363 (1973)
84. George, T., Saxena, V., Gupta, A., Singh, A., Choudhury, A.: Multifrontal factorization of sparse spd matrices on GPUs. In: Proc. of IEEE International Parallel and Distributed Processing Symposium (IPDPS 2011). Anchorage, Alaska (2011)
85. Ghysels, P., Li, X.S., Gorman, C., Rouet, F.H.: A robust parallel preconditioner for indefinite systems using hierarchical matrices and randomized sampling. In: 2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 897–906. IEEE (2017)
86. Ghysels, P., Li, X.S., Rouet, F.H., Williams, S., Napov, A.: An efficient multicore implementation of a novel HSS-structured multifrontal solver using randomized sampling. SIAM J. Sci. Comput. **38**(5), S358–S384 (2016)
87. Ghysels, P., Synk, R.: High performance sparse multifrontal solvers on modern GPUs. Parallel Computing **110**, 102897 (2022)
88. Gilbert, J.R., Tarjan, R.E.: The analysis of a nested dissection algorithm. Numerische Mathematik pp. 377–404 (1987)
89. Gillman, A., Martinsson, P.G.: An $O(N)$ algorithm for constructing the solution operator to 2D elliptic boundary value problems in the absence of body loads. Advances in Computational Mathematics **40**(4), 773–796 (2014)
90. Gillman, A., Young, P.M., Martinsson, P.G.: A direct solver with $O(N)$ complexity for integral equations on one-dimensional domains. Frontiers of Mathematics in China **7**(2), 217–247 (2012)
91. Gorman, C., Chávez, G., Ghysels, P., Mary, T., Rouet, F.H., Li, X.S.: Robust and accurate stopping criteria for adaptive randomized sampling in matrix-free hierarchically semiseparable construction. SIAM Journal on Scientific Computing **41**(5), S61–S85 (2019)
92. Grasedyck, L., Hackbusch, W.: Construction and arithmetics of H-matrices. Computing **70**(4), 295–334 (2003)
93. Grasedyck, L., Kriemann, R., Le Borne, S.: Parallel black box $\mathcal{H}$ -lu preconditioning for elliptic boundary value problems. Computing and visualization in science **11**(4), 273–291 (2008)
94. Guo, H., Hu, J., Michielssen, E.: On MLMDA/butterfly compressibility of inverse integral operators. IEEE Antennas Wirel. Propag. Lett. **12**, 31–34 (2013)
95. Guo, H., Hu, J., Nie, Z.: An MPI-OpenMP hybrid parallel $\mathcal{H}$ -LU direct solver for electromagnetic integral equations. International Journal of Antennas and Propagation **2015**(1), 615743 (2015)
96. Guo, H., Liu, Y., Hu, J., Michielssen, E.: A butterfly-based direct integral-equation solver using hierarchical LU factorization for analyzing scattering from electrically large conducting objects. IEEE Trans. Antennas Propag. **65**(9), 4742–4750 (2017)
97. Gupta, A.: Recent advances in direct methods for solving unsymmetric sparse systems of linear equations. ACM Trans. Math. Softw. **28**(3), 301–324 (2002). doi:10.1145/569147.569149
98. Gupta, A., Karypis, G., Kumar, V.: Highly scalable parallel algorithms for sparse matrix factorization. IEEE Transactions on Parallel and Distributed Systems **8**(5), 502–520 (1997). doi:10.1109/71.598277

99. H2Opus. <https://github.com/ecrc/h2opus>
100. Hackbusch, W., Börm, S.: H2-matrix approximation of integral operators by interpolation. *Applied numerical mathematics* **43**(1-2), 129–143 (2002)
101. Hao, S., Martinsson, P.G.: A direct solver for elliptic PDEs in three dimensions based on hierarchical merging of Poincaré–Steklov operators. *Journal of Computational and Applied Mathematics* **308**, 419–434 (2016)
102. Heldring, A., Ubada, E., Rius, J.: Fast direct high frequency MoM solver using butterfly algebra. *Authorea Preprints*
103. Hennessy, J., Patterson, D., Kozyrakis, C.: *Computer Architecture: A Quantitative Approach*. The Morgan Kaufmann Series in Computer Architecture and Design. Elsevier Science (2025). URL <https://books.google.com/books?id=JVfY0AEACAAJ>
104. Hénon, P., Ramet, P., Roman, J.: PaStiX: a high-performance parallel direct solver for sparse symmetric positive definite systems. *Parallel Computing* **28**(2), 301–321 (2002)
105. HiCMA. <https://github.com/ecrc/hicma>
106. HLibPro. <https://www.hlibpro.com>
107. Ho, K.L., Ying, L.: Hierarchical interpolative factorization for elliptic operators: differential equations. *Communications on Pure and Applied Mathematics* **69**(8), 1415–1451 (2016)
108. Huang, H., Xing, X., Chow, E.: H2Pack: High-performance H2 matrix package for kernel matrices using the proxy point method. *ACM Transactions on Mathematical Software (TOMS)* **47**(1), 1–29 (2020)
109. Huang, H., Xu, T., Xi, Y., Chow, E.: HiGP: A high-performance Python package for Gaussian process. *arXiv preprint arXiv:2503.02259* (2025)
110. Irony, D., Toledo, S.: Trading replication for communication in parallel distributed-memory dense solvers. *Parallel Processing Letters* **12**(01), 79–94 (2002)
111. Kale, L.V., Zheng, G.: Charm++ and AMPI: adaptive runtime strategies via migratable objects, chap. 13, pp. 265–282. John Wiley & Sons, Ltd (2009). doi:<https://doi.org/10.1002/9780470558027.ch13>
112. Kandalla, K., Knaak, D., McMahon, K., Radcliffe, N., Pagel, M.: Optimizing Cray MPI and Cray SHMEM for current and next generation Cray-XC supercomputers. *Cray User Group (CUG)* **2015** (2015)
113. Kashi, A., Nayak, P., Kulkarni, D., Scheinberg, A., Lin, P., Anzt, H.: Batched sparse iterative solvers on GPU for the collision operator for fusion plasma simulations. In: *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 157–167 (2022). doi:10.1109/IPDPS53621.2022.00024
114. KBLAS-GPU. <https://github.com/ecrc/kblas-gpu>
115. Keyes, D., Ltaief, H., Turkiyyah, G.: Hierarchical algorithms on hierarchical architectures. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* **378**(2166), 20190055 (2020). doi:10.1098/rsta.2019.0055
116. Khalichi, B., Cerfon, A., Malhotra, D., O’Neil, M.: Taylor relaxations with stepped-pressure profile: integral equation solver (BIEST) for computation of multi-region relaxed MHD equilibria. In: *2025 International Applied Computational Electromagnetics Society Symposium (ACES)*, pp. 1–2. IEEE (2025)
117. Khan, R., Kandappan, V., Ambikasaran, S.: HODLRdD: A new black-box fast algorithm for N-body problems in d-dimensions with guaranteed error bounds: Applications to integral equations and support vector machines. *J. Comput. Phys.* **501**, 112786 (2024)
118. Kielstra, P.M., Shi, T., Luo, H., Qian, J., Liu, Y.: A linear-complexity tensor butterfly algorithm for compressing high-dimensional oscillatory integral operators. *SIAM Multiscale Model. Simul.* **23**(2), 864–893 (2025)
119. Kokkos Kernels. <https://github.com/kokkos/kokkos-kernels>
120. Kressner, D., Susnjara, A.: Fast QR decomposition of HODLR matrices. *arXiv preprint arXiv:1809.10585* (2018)
121. Kriemann, R.: H-LU factorization on many-core systems. *Computing and Visualization in Science* **16**(3), 105–117 (2013)
122. Kriemann, R.: Hierarchical low-rank arithmetic with floating point compression. *SIAM Journal on Scientific Computing* **47**(4), B763–B784 (2025)
123. Kump, J., Yesypenko, A., Martinsson, P.G.: A two-level direct solver for the hierarchical Poincaré–Steklov method. *arXiv preprint arXiv:2503.04033* (2025)
124. L. Ho, K., Ying, L.: Hierarchical interpolative factorization for elliptic operators: integral equations. *Communications on Pure and Applied Mathematics* **69**(7), 1314–1353 (2016)
125. Lacoste, X., Faverge, M., Bosilca, G., Ramet, P., Thibault, S.: Taking advantage of hybrid systems for sparse direct solvers via task-based runtimes. In: *2014 IEEE International Parallel & Distributed Processing Symposium Workshops*, pp. 29–38 (2014). doi:10.1109/IPDPSW.2014.9. URL <https://starp.u.giitlabpages.inria.fr/>
126. Levitt, J., Martinsson, P.G.: Linear-complexity black-box randomized compression of rank-structured matrices. *SIAM Journal on Scientific Computing* **46**(3), A1747–A1763 (2024)
127. Levitt, J., Martinsson, P.G.: Randomized compression of rank-structured matrices accelerated with graph coloring. *Journal of Computational and Applied Mathematics* **451**, 116044 (2024)
128. Li, X.: Factorization-based sparse solvers and preconditioners. In: Z. Bai, W. Gao, Y. Su (eds.) *Series in Contemporary Applied Mathematics: Vol 19, Matrix Functions and Matrix Equations*, pp. 109–137. World Scientific (2015). doi:10.1142/9789814675772_0005. Extended Summary for 4th Gene Golub SIAM Summer School, July 22–Aug. 9, 2013, Shanghai
129. Li, X.S., Demmel, J.: Making sparse Gaussian elimination scalable by static pivoting. In: *SC ’98: Proceedings of the 1998 ACM/IEEE Conference on Supercomputing*, pp. 34–34 (1998). doi:10.1109/SC.1998.10030
130. Li, X.S., Demmel, J.W.: SuperLU-DIST: A scalable distributed-memory sparse direct solver for unsymmetric linear systems. *ACM Trans. Math. Softw.* **29**(2), 110–140 (2003). <https://portal.nersc.gov/project/sparse/superlu/>
131. Li, Y., Liu, J.: Hierarchical Tucker low-rank matrices: construction and matrix-vector multiplication. *arXiv preprint arXiv:2508.05958* (2025)
132. Li, Y., Poulson, J., Ying, L.: Distributed-memory $\mathcal{H}$ -matrix algebra I: data distribution and matrix-vector multiplication. *arXiv preprint arXiv:2008.12441* (2020)
133. Li, Y., Yang, H., Martin, E.R., Ho, K.L., Ying, L.: Butterfly factorization. *SIAM Multiscale Model. Simul.* **13**(2), 714–732 (2015)

134. Li, Y., Ying, L.: Distributed-memory hierarchical interpolative factorization. *Research in the Mathematical Sciences* **4**(1), 12 (2017)
135. Li, Xiaoye S. and Lin, Paul and Liu, Yang and Sao, Piyush: Newly released capabilities in the distributed-memory SuperLU sparse direct solver. *ACM Trans. Math. Softw.* **49**(1) (2023). doi:10.1145/3577197
136. Liang, T., Chen, C., Martinsson, P.G., Biros, G.: An $O(N)$ distributed-memory parallel direct solver for planar integral equations. In: 2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 440–452. IEEE (2024). doi:10.1109/IPDPS57955.2024.00046
137. Liberty, E., Woolfe, F., Martinsson, P.G., Rokhlin, V., Tytgert, M.: Randomized algorithms for the low-rank approximation of matrices. *Proceedings of the National Academy of Sciences* **104**(51), 20167–20172 (2007). doi:10.1073/pnas.0709640104. URL <https://www.pnas.org/doi/abs/10.1073/pnas.0709640104>
138. Liegeois, K., Rajamanickam, S., Berger-Vergiat, L.: Performance portable batched sparse linear solvers. *IEEE Transactions on Parallel and Distributed Systems* **34**(5), 1524–1535 (2023). doi:10.1109/TPDS.2023.3249110
139. Lin, L., Lu, J., Ying, L.: Fast construction of hierarchical matrix representation from matrix–vector multiplication. *J. Comput. Phys.* **230**(10), 4071–4087 (2011)
140. Lindstrom, P., Hittinger, J., Diffenderfer, J., Fox, A., Osei-Kuffuor, D., Banks, J.: ZFP: A compressed array representation for numerical computations. *The International Journal of High Performance Computing Applications* **39**(1), 104–122 (2025)
141. Lindstrom, P., Isenburt, M.: Fast and efficient compression of floating-point data. *IEEE transactions on visualization and computer graphics* **12**(5), 1245–1250 (2006)
142. Lipton, R.J., Rose, D.J., Tarjan, R.E.: Generalized nested dissection. *SIAM J. Numerical Analysis* **16**, 346–358 (1979)
143. Litvinenko, A., Kriemann, R., Genton, M.G., Sun, Y., Keyes, D.E.: HLIBCov: Parallel hierarchical matrix approximation of large covariance matrices and likelihoods with applications in parameter identification. *MethodsX* **7**, 100600 (2020)
144. Liu, J.W.H.: The role of elimination trees in sparse factorization. *SIAM J. Matrix Anal. Appl.* **11**(1), 134–172 (1990)
145. Liu, W., Li, A., Hogg, J., Duff, I.S., Vinter, B.: A synchronization-free algorithm for parallel sparse triangular solves. In: *Proceedings of the 22nd International Conference on Euro-Par 2016: Parallel Processing - Volume 9833*, p. 617–630. Springer-Verlag, Berlin, Heidelberg (2016). doi:10.1007/978-3-319-43659-3_45. URL https://doi.org/10.1007/978-3-319-43659-3_45
146. Liu, Y.: ButterflyPACK (2018). <https://portal.nersc.gov/project/sparse/butterflypack/>
147. Liu, Y., Ding, N., Sao, P., Williams, S., Li, X.S.: Unified communication optimization strategies for sparse triangular solver on CPU and GPU clusters. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '23*. Association for Computing Machinery, New York, NY, USA (2023). doi:10.1145/3581784.3607092
148. Liu, Y., Ghysels, P., Claus, L., Li, X.S.: Sparse approximate multifrontal factorization with butterfly compression for high-frequency wave equations. *SIAM Journal on Scientific Computing* **43**(5), S367–S391 (2021)
149. Liu, Y., Guo, H., Michielssen, E.: An HSS matrix-inspired butterfly-based direct solver for analyzing scattering from two-dimensional objects. *IEEE Antennas Wirel. Propag. Lett.* **16**, 1179–1183 (2017)
150. Liu, Y., Jacquelin, M., Ghysels, P., Li, X.: Highly scalable distributed-memory sparse triangular solution algorithms. In: *Proceedings of the SIAM Workshop on Combinatorial Scientific Computing*. Bergen, Norway (2018)
151. Liu, Y., Luo, T., Rani, A., Luo, H., Li, X.S.: Detecting resonance of radio-frequency cavities using fast direct integral equation solvers and augmented Bayesian optimization. *IEEE J. Multiscale Multiphysics Comput. Tech.* (2023)
152. Liu, Y., Shi, T., Kielstra, P.M., Luo, H., Qian, J.: A block diagonal preconditioned and tensor butterfly accelerated volume integral equation solver for 3D high-frequency scattering analysis. In: *2025 International Applied Computational Electromagnetics Society Symposium (ACES)*, pp. 1–1. IEEE (2025)
153. Liu, Y., Sid-Lakhdar, W., Rebrova, E., Ghysels, P., Li, X.S.: A parallel hierarchical blocked adaptive cross approximation algorithm. *Int. J. High Perform. C.* **0**(0), 1094342020918305 (0). doi:10.1177/1094342020918305
154. Liu, Y., Song, J., Burrridge, R., Qian, J.: A fast butterfly-compressed Hadamard-Babich integrator for high-frequency Helmholtz equations in inhomogeneous media with arbitrary sources. *SIAM Multiscale Model. Simul.* **21**(1), 269–308 (2023)
155. Liu, Y., Xing, X., Guo, H., Michielssen, E., Ghysels, P., Li, X.: Butterfly factorization via randomized matrix-vector multiplications. *SIAM J. Sci. Comput.* **43**(2), A883–A907 (2021). doi:10.1137/20M1315853
156. Lucas, R., Wagenbreth, G., Davis, D., Grimes, R.: Multifrontal computations on GPUs and their multi-core hosts. In: *VECPAR'10: Proc. 9th Intl. Meeting for High Performance Computing for Computational Science*. Berkeley, CA (2010). <http://vecpar.fe.up.pt/2010/papers/5.php>
157. Ma, M., Jiao, D.: Accuracy directly controlled fast direct solution of general $\mathcal{H}^2$ -matrices and its application to solving electrodynamic volume integral equations. *IEEE Transactions on Microwave Theory and Techniques* **66**(1), 35–48 (2017)
158. Ma, M., Jiao, D.: Direct solution of general $\mathcal{H}^2$ -matrices with controlled accuracy and concurrent change of cluster bases for electromagnetic analysis. *IEEE Transactions on Microwave Theory and Techniques* **67**(6), 2114–2127 (2019). doi:10.1109/TMTT.2019.2914391
159. Ma, Q., Deshmukh, S., Yokota, R.: Scalable linear time dense direct solver for 3-d problems without trailing sub-matrix dependencies. In: *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–12. IEEE (2022)
160. Ma, Q., Yokota, R.: An inherently parallel $\mathcal{H}^2$ -ULV factorization for solving dense linear systems on GPUs. *The International Journal of High Performance Computing Applications* **38**(4), 314–336 (2024). doi:10.1177/10943420241242021
161. Manić, A.B., Smull, A.P., Rouet, F.H., Li, X.S., Notaroš, B.M.: Efficient scalable parallel higher order direct MoM-SIE method with hierarchically semiseparable structures for 3-D scattering. *IEEE Trans. Antennas Propag.* **65**(5), 2467–2478 (2017)
162. Martinsson, P.G.: A fast randomized algorithm for computing a hierarchically semiseparable representation of a matrix. *SIAM Journal on Matrix Analysis and Applications* **32**(4), 1251–1274 (2011)
163. Martinsson, P.G.: A direct solver for variable coefficient elliptic PDEs discretized via a composite spectral collocation method. *J. Comput. Phys.* **242**, 460–479 (2013)
164. Martinsson, P.G., O’Neil, M.: Fast direct solvers. *arXiv preprint arXiv:2511.07773* (2025)
165. Martinsson, P.G., Quintana-Ortí, G., Heavner, N.: RandUTV: a blocked randomized algorithm for computing a rank-revealing UTV factorization. *ACM Trans. Math. Softw.* **45**(1) (2019). doi:10.1145/3242670. URL <https://doi.org/10.1145/3242670>

166. Martinsson, P.G., Rokhlin, V.: A fast direct solver for boundary integral equations in two dimensions. *J. Comput. Phys.* **205**(1), 1–23 (2005)
167. Mary, T.: Block low-rank multifrontal solvers: complexity, performance, and scalability. Ph.D. thesis, Université Paul Sabatier-Toulouse III (2017)
168. Michielssen, E., Boag, A.: A multilevel matrix decomposition algorithm for analyzing scattering from large structures. *IEEE Trans. Antennas Propag.* **44**(8), 1086–1093 (1996)
169. Minden, V., Damle, A., Ho, K.L., Ying, L.: Fast spatial Gaussian process maximum likelihood estimation via skeletonization factorizations. *SIAM Multiscale Model. Simul.* **15**(4), 1584–1611 (2017)
170. Minden, V., Ho, K.L., Damle, A., Ying, L.: A recursive skeletonization factorization based on strong admissibility. *SIAM Multiscale Model. Simul.* **15**(2), 768–796 (2017). URL <https://doi.org/10.1137/16M1095949>
171. Private Coommunication (2026)
172. MUMPS. <http://mumps.enseeiht.fr/>
173. mxc. <https://github.com/QxGbM/mxc>
174. Nguyen, C., Boyko, A., Okhmatovski, V.: Tensor train accelerated method of moment solution of volume integral equations for arbitrary objects with logarithmic complexity. *IEEE Trans. Microw. Theory Tech.* **73**(12) (2025)
175. Nies, R., Hoelzl, M.: Testing performance with and without block low rank compression in MUMPS and the new PaStiX 6.0 for JOREK nonlinear MHD simulations. *arXiv:1907.13442* (2019)
176. Nuca, R., Parsani, M., Turkiyyah, G.: An adaptive two-stage algorithm for error-bounded scientific data compression. In: 2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 987–997. IEEE (2025)
177. Pang, Q., Ho, K.L., Yang, H.: Interpolative decomposition butterfly factorization. *SIAM J. Sci. Comput.* **42**(2), A1097–A1115 (2020)
178. PanguLU. <https://github.com/SuperScientificSoftwareLaboratory/PanguLU>
179. PARDISO. <https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-c/2023-0/pardiso.html>
180. Park, J., Smelyanskiy, M., Sundaram, N., Dubey, P.: Sparsifying synchronization for high-performance shared-memory sparse triangular solver. In: J.M. Kunkel, T. Ludwig, H.W. Meuer (eds.) *Supercomputing*, pp. 124–140. Springer International Publishing, Cham (2014)
181. PaStiX. <https://gitlab.inria.fr/solverstack/pastix>
182. Pearce, K.J., Yesypenko, A., Levitt, J., Martinsson, P.G.: Randomized rank-structured matrix compression by tagging. *arXiv preprint arXiv:2501.05528* (2025)
183. Poulson, J., Demanet, L., Maxwell, N., Ying, L.: A parallel butterfly algorithm. *SIAM Journal on Scientific Computing* **36**(1), C49–C65 (2014)
184. Pouransari, H., Coulier, P., Darve, E.: Fast hierarchical solvers for sparse matrices using extended sparsification and low-rank approximation. *SIAM J. Sci. Comput.* **39**(3), A797–A830 (2017). doi:10.1137/15M1046939
185. Rebrova, E., Chávez, G., Liu, Y., Ghysels, P., Li, X.S.: A study of clustering techniques and hierarchical matrix formats for kernel ridge regression. In: 2018 IEEE international parallel and distributed processing symposium workshops (IPDPSW), pp. 883–892. IEEE (2018)
186. Ren, J., Zhong, T., Hong, Y., Feng, G., Wang, X., Jia, W., Ltaief, H., Keyes, D.E.: Caracal: A GPU-resident sparse LU solver with lightweight fine-grained scheduling. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '25*, p. 1477–1494. Association for Computing Machinery, New York, NY, USA (2025). doi:10.1145/3712285.3759792
187. Rouet, F.H., Li, X.S., Ghysels, P., Napov, A.: A distributed-memory package for dense hierarchically semi-separable matrix computations using randomization. *ACM Transactions on Mathematical Software (TOMS)* **42**(4), 1–35 (2016)
188. Sao, P., Kannan, R., Li, X.S., Vuduc, R.: A communication-avoiding 3d sparse triangular solver. In: *Proceedings of the ACM International Conference on Supercomputing, ICS '19*, p. 127–137. Association for Computing Machinery, New York, NY, USA (2019). doi:10.1145/3330345.3330357
189. Sao, P., Li, X.S.: Brief announcement: communication optimal sparse LU factorization for planar matrices. *SPAA '23*, p. 427–430. Association for Computing Machinery, New York, NY, USA (2023). doi:10.1145/3558481.3591315
190. Sao, P., Liu, X., Vuduc, R., Li, X.: A sparse direct solver for distributed memory Xeon Phi-accelerated systems. In: 29th IEEE International Parallel & Distributed Processing Symposium (IPDPS). Hyderabad, India (2015)
191. Sao, P., Vuduc, R., Li, X.: A distributed CPU-GPU sparse direct solver. In: *Proc. of Euro-Par 2014, LNCS Vol. 8632*, pp. 487–498. Porto, Portugal (2014)
192. Sao, P., Vuduc, R., Li, X.: A communication-avoiding 3D algorithm for sparse LU factorization on heterogeneous systems. *J. Parallel and Distributed Computing* (2019). doi:10.1016/j.jpdc.2019.03.004. <https://www.sciencedirect.com/science/article/abs/pii/S0743731518305197>
193. Sayed, S.B., Liu, Y., Gomez, L.J., Yucel, A.C.: A butterfly-accelerated volume integral equation solver for broad permittivity and large-scale electromagnetic analysis. *IEEE Trans. Antennas Propag.* **70**(5), 3549–3559 (2022)
194. Shaeffer, J.: Direct solve of electrically large integral equations for problem sizes to 1 M unknowns. *IEEE Trans. Antennas Propag.* **56**(8), 2306–2313 (2008)
195. Shen, J., Wang, Y., Xia, J.: Fast structured direct spectral methods for differential equations with variable coefficients, I. the one-dimensional case. *SIAM J. Sci. Comput.* **38**(1), A28–A54 (2016). doi:10.1137/140986815
196. Solomonik, E., Demmel, J.: Communication-optimal parallel 2.5D matrix multiplication and LU factorization algorithms. In: *Euro-Par 2011 Parallel Processing*, pp. 90–109. Springer (2011)
197. Spendthofer, T., Ma, Q., Matsumoto, Y., Yokota, R.: On the interplay between precision, rank and admissibility for iterative hierarchical low-rank matrix solvers. In: *ISC High Performance 2025 Research Paper Proceedings (40th International Conference)*, pp. 1–12. Prometheus GmbH (2025)

198. STRUMPACK: STRUctured Matrices PACKages (last accessed 2025). <http://portal.nersc.gov/project/sparse/strumpack/>
199. SuperLU: sparse direct solver (Last accessed: 2025). <http://portal.nersc.gov/project/sparse/superlu/>
200. Sushnikova, D., Greengard, L., O'Neil, M., Rachh, M.: FMM-LU: a fast direct solver for multiscale boundary integral equations in three dimensions. *SIAM Multiscale Model. Simul.* **21**(4), 1570–1601 (2023). doi:10.1137/22M1514040
201. symPACK. <https://github.com/symPACK/symPACK>
202. Totoni, E., Heath, M.T., Kale, L.V.: Structure-adaptive parallel solution of sparse triangular linear systems. *Parallel Comput.* **40**(9), 454–470 (2014). doi:10.1016/j.parco.2014.06.006
203. Vuduc, R., Chandramowlishwaran, A., Choi, J., Guney, M., Shringarpure, A.: On the limits of GPU acceleration. In: Proc. of the 2nd USENIX conference on Hot topics in parallelism, HotPar'10. Berkeley, CA (2010)
204. Wang, S., de Hoop, M.V., Xia, J.: On 3D modeling of seismic wave propagation via a structured parallel multifrontal direct Helmholtz solver. *Geophysical Prospecting* **59**(Modelling Methods for Geophysical Imaging: Trends and Perspectives), 857–873 (2011)
205. Wang, S., de Hoop, M.V., Xia, J., Li, X.S.: Massively parallel structured multifrontal solver for time-harmonic elastic waves in 3-D anisotropic media. *Geophys. J. Int.* **191**(1), 346–366 (2012)
206. Wang, S., Li, X.S., Rouet, F.H., Xia, J., De Hoop, M.V.: A parallel geometric multifrontal solver using hierarchically semiseparable structure. *ACM Trans. Math. Softw.* **42**(3) (2016). doi:10.1145/2830569. URL <https://doi.org/10.1145/2830569>
207. Wang, X., Liu, W., Xue, W., Wu, L.: swSpTRSV: a fast sparse triangular solve with sparse level tile layout on sunway architectures. *SIGPLAN Not.* **53**(1), 338–353 (2018). doi:10.1145/3200691.3178513
208. Wolfgang, H.: Hierarchical matrices: Algorithms and analysis (2015)
209. WSMP. <https://dominoweb.draco.res.ibm.com/95ce158b896a010d85256a57006aadaaf.html>
210. Xi, Y., Xia, J.: On the stability of some hierarchical rank structured matrix algorithms. *SIAM Journal on Matrix Analysis and Applications* **37**(3), 1279–1303 (2016)
211. Xia, J.: Randomized sparse direct solvers. *SIAM Journal on Matrix Analysis and Applications* **34**(1), 197–227 (2013)
212. Xia, J.: $O(N)$ complexity randomized 3D direct solver with HSS2D structure. *Proceedings of the Project Review, Geo-Mathematical Imaging Group* pp. 317–325 (2014)
213. Xia, J.: Making the Nyström method highly accurate for low-rank approximations. *SIAM Journal on Scientific Computing* **46**(2), A1076–A1101 (2024). doi:10.1137/23M1585039
214. Xia, J., Chandrasekaran, S., Gu, M., Li, X.S.: Fast algorithms for hierarchically semiseparable matrices. *Numerical Linear Algebra with Applications* **17**(6), 953–976 (2010)
215. Xia, J., Chandrasekaran, S., Gu, M., Li, X.S.: Superfast multifrontal method for large structured linear systems of equations. *SIAM Journal on Matrix Analysis and Applications* **31**(3), 1382–1411 (2010)
216. Xia, J.Z.: Effective and robust preconditioning of general SPD matrices via structured incomplete factorization. *SIAM Journal on Matrix Analysis and Applications* **38**(4), 1298–1322 (2017)
217. Xiao, J., Gu, M., Langou, J.: Fast parallel randomized QR with column pivoting algorithms for reliable low-rank matrix approximations. In: 2017 IEEE 24th International Conference on High Performance Computing (HiPC), pp. 233–242 (2017). doi:10.1109/HiPC.2017.00035
218. Xing, X., Chow, E.: Interpolative decomposition via proxy points for kernel matrices. *SIAM Journal on Matrix Analysis and Applications* **41**(1), 221–243 (2020)
219. Yamazaki, I., Ida, A., Yokota, R., Dongarra, J.: Distributed-memory lattice H-matrix factorization. *The International Journal of High Performance Computing Applications* **33**(5), 1046–1063 (2019)
220. Ye, X., Xia, J., Ying, L.: Analytical low-rank compression via proxy point selection. *SIAM Journal on Matrix Analysis and Applications* **41**(3), 1059–1085 (2020)
221. Yesypenko, A., Chen, C., Martinsson, P.G.: A simplified fast multipole method based on strong recursive skeletonization. *J. Comput. Phys.* **524**, 113707 (2025)
222. Yesypenko, A., Martinsson, P.G.: SlabLU: a two-level sparse direct solver for elliptic PDEs. *Advances in Computational Mathematics* **50**(4), 90 (2024)
223. Yesypenko, A., Martinsson, P.G.: Randomized strong recursive skeletonization: simultaneous compression and LU factorization of hierarchical matrices using matrix-vector products (2025)
224. Ying, L.: A pedestrian introduction to fast multipole methods. *Science China Mathematics* **55**(5), 1043–1051 (2012)
225. Ying, L.: Directional preconditioner for 2D high frequency obstacle scattering. *SIAM Multiscale Model. Simul.* **13**(3), 829–846 (2015). doi:10.1137/140985135
226. Yu, C., Wang, W., Pierce, D.: A CPU-GPU hybrid approach for the unsymmetric multifrontal method. *Parallel Computing* **37**, 759–770 (2011)
227. Zampini, S., Boukaram, W., Turkiyyah, G., Knio, O., Keyes, D.: H2Opus: a distributed-memory multi-GPU software package for non-local operators. *Advances in Computational Mathematics* **48**(3), 31 (2022)
228. Zhao, K., Vouvakis, M.N., Lee, J.F.: The adaptive cross approximation algorithm for accelerated method of moments computations of EMC problems. *IEEE Transactions on Electromagnetic Compatibility* **47**(4), 763–773 (2005). doi:10.1109/TEM.2005.857898
229. Zhao, Y., Jiao, D., Mao, J.: Fast nested cross approximation algorithm for solving large-scale electromagnetic problems. *IEEE Transactions on Microwave Theory and Techniques* **67**(8), 3271–3283 (2019)
230. Zhou, B., Jiao, D.: Direct finite-element solver of linear complexity for large-scale 3-D electromagnetic analysis and circuit extraction. *IEEE T. MICROW. THEORY* **63**(10), 3066–3080 (2015)
231. Zhu, X., Lai, J.: Recursive sparse LU decomposition based on nested dissection and low rank approximations. *J. Comput. Phys.* **539**, 114231 (2025)